

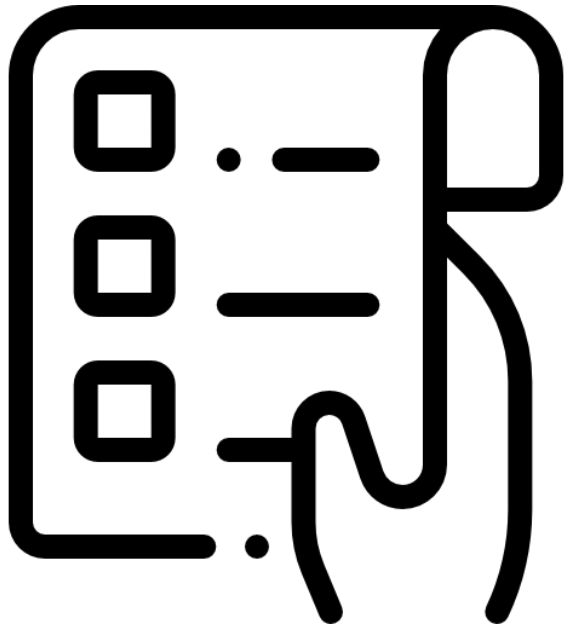
Белорусско-Российский университет
Кафедра «Программное обеспечение
информационных технологий»

ЭВМ, периферийные устройства и контроллеры

**Тема: Архитектурные
особенности организации
ЭВМ различных классов**

Кутузов Виктор Владимирович
Республика Беларусь, Могилев, 2025





Содержание лекции

Содержание лекции

Тема: Архитектурные особенности организации ЭВМ различных классов

1. [Рекомендуемые материалы по теме](#)
2. [Архитектура и организация компьютера](#)
3. [Классические архитектуры ЭВМ](#)
4. [Архитектура Джона фон Неймана](#)
5. [Гарвардская архитектура](#)
6. [Архитектура фон Неймана vs Гарвардская архитектура](#)
7. [Классификация Архитектур ЭВМ.](#)
8. [- по организации памяти и кода](#)
9. [- по модели параллелизма \(классификация Флинна и далее\)](#)
10. - по способу организации процессора (микроархитектура)

Содержание лекции

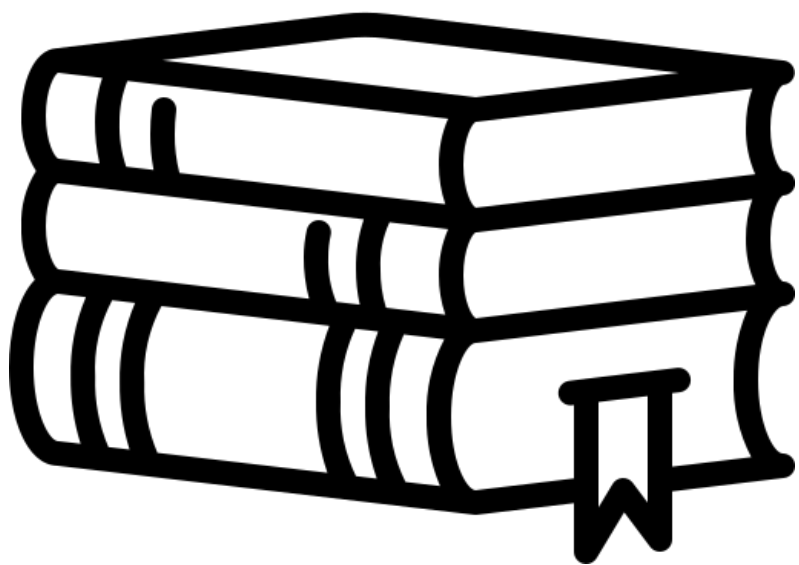
Тема: Архитектурные особенности организации ЭВМ различных классов

11. - по типу набора инструкций (ISA)
12. -- RISC \ CISC архитектуры
13. - по организации данных и вычислений
14. - по памяти и иерархии хранения
15. - по вводу-выводу и шинам
16. - по организации многопроцессорности
17. - по способу адресации и доступу к памяти
18. - по безопасности и изоляции
19. - по устойчивости и надёжности
20. - исторические и учебные архитектуры

Содержание лекции

Тема: Архитектурные особенности организации ЭВМ различных классов

21. - по специализированным областям применения



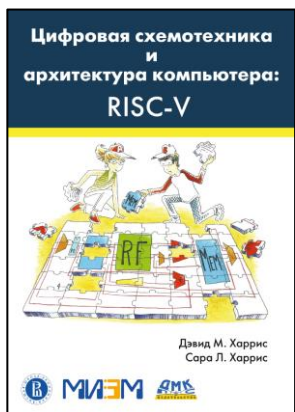
Рекомендуемые материалы по теме



Литература



Орлов С. А., Цилькер Б. Я. **Организация ЭВМ и систем:** Учебник для вузов. 2-е изд. – СПб.: Питер, 2011. – 688 с.
<https://rutracker.org/forum/viewtopic.php?t=5310884>



Сара Л. Харрис, Дэвид Харрис **Цифровая схемотехника и архитектура компьютера: RISC-V** / пер. с англ. В. С. Яценкова, А. Ю. Романова; под ред. А. Ю. Романова. – М.: ДМК Пресс, 2021. – 810 с. <https://rutracker.org/forum/viewtopic.php?t=6204850>



Архитектура и организация компьютера



Архитектура и организация компьютера

- Под архитектурой компьютера в общем обычно понимается набор типов данных, операций и характеристик каждого отдельно взятого уровня ПК.
- Она связана с теми аспектами, что видимы программирующему пользователю соответствующего уровня.
- Например, ресурсы памяти – это аспект архитектуры. А технология реализации запоминающих устройств, физические принципы и механизмы их работы, хотя и являются базовыми, но к архитектуре компьютера в этом смысле не относятся.
- Но при анализе понятия архитектура ПК нужно иметь в виду, что на сегодня разработка компьютера – это не независимая разработка «железа» (hardware) и программного обеспечения (software) по отдельности, но их разработка в едином комплексе.
- Понятие архитектуры актуально в контексте взаимодействия аппаратных и программных компонентов.

Стандартный цикл работы компьютера

- Можно выделить условный стандартный цикл работы компьютера:
 - 1. выборка инструкции из основной памяти;
 - 2. выборка операндов из оперативной памяти и/или регистров;
 - 3. выполнение инструкции;
 - 4. фиксация результата – запись в память, регистр, вывод.

Архитектура ПК

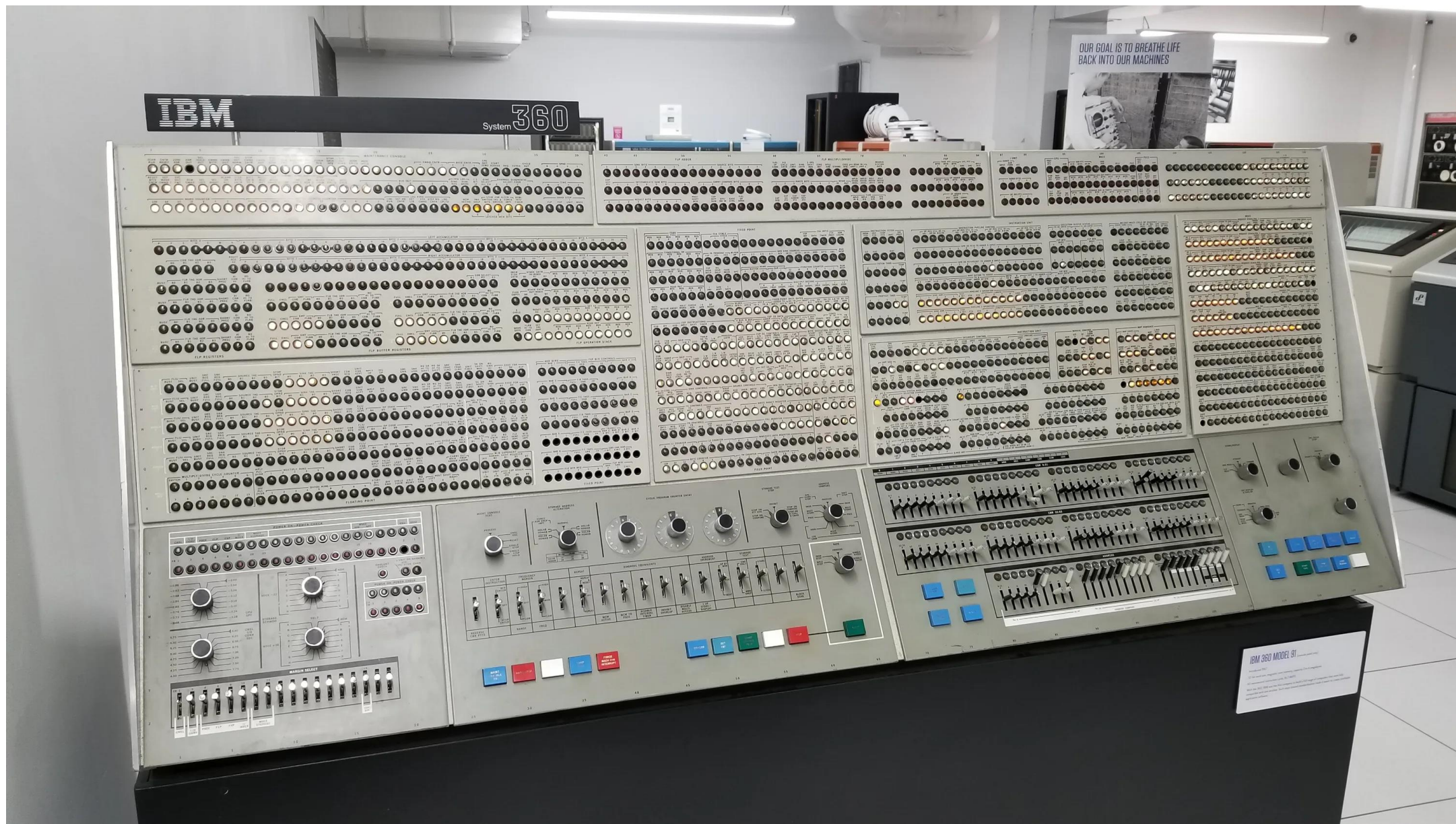
- На сегодня существует ряд трактовок и определений понятия архитектуры.
- Термин «архитектура» применительно к компьютерам впервые был использован корпорацией IBM в середине 60-х годов при разработке семейства ПК IBM 360.
- Под термином «архитектура» часто понимается представление возможностей компьютера с точки зрения пользователя, разрабатывающего программу на машинно-ориентированном языке.
- Соответственно, архитектура отражает те стороны структуры и функционирования компьютера, что существенны и видимы для программиста в контексте разрабатываемых им программ.



IBM System/360



IBM System/360



Архитектура IBM System/360

- **IBM System/360** – это семейство мейнфреймов, представленное IBM в 1964 году.
- Оно стало революционным в истории вычислительной техники, поскольку впервые предложило единую архитектуру для различных по производительности и назначению компьютеров.
- **Проблема, которую оно решало:**
 - **До System/360 IBM выпускала несовместимые линейки компьютеров** (1400-я серия для бизнеса, 7000-я – для науки).
 - Клиенты вынуждены были полностью переписывать ПО при переходе на новую модель.
- **Революционная идея:**
- **IBM System/360 ввела понятие единой архитектуры для всей линейки машин** – от недорогих моделей для учётных задач до суперкомпьютеров. Это означало:
 - Одно и то же ПО работало на всех моделях (с разной скоростью),
 - Аппаратное расширение без замены ПО,
 - Стандартизация интерфейсов и периферии.
- **Название "360": Обозначало "полный круг" – архитектура покрывала все возможные сценарии использования.**

Архитектура ПК. Термины

- **Архитектура** – совокупность характеристик компьютера, существенных с точки зрения пользователя. Можно видеть, что ключевой момент здесь – способ использования компьютера.
- **Архитектура** – совокупность общих принципов организации аппаратно-программных средств и их характеристик, определяющая функциональные возможности вычислителя при решении соответствующих классов задач.
- **Архитектура** – логическое построение вычислителя, как оно представляется программисту, разрабатывающему программу на машинно-ориентированном языке.
- **Архитектура компьютера** – это модель компьютерной системы, воплощённая в её компонентах, их взаимодействии между собой и окружением, включающая также принципы её проектирования и развития. Аспекты реализации (например, технология, применяемая при реализации памяти) не являются частью архитектуры

Архитектура ПК

- **Архитектура связана с аспектами, которые видны программисту.**
 - **Например**, сведения о том, сколько памяти можно использовать при написании программы – часть архитектуры, а аспекты разработки (например, какая технология используется при создании памяти) не является частью архитектуры.
- Изучение того, как разрабатываются те части компьютерной системы, которые видны программистам, называется изучением компьютерной архитектуры.
- Термины компьютерная архитектура и компьютерная организация означают, в сущности, одно и то же...”
- **Архитектура может трактоваться** также как раскрытие способов взаимодействия центрального процессора (ЦП) с памятью, периферийными устройствами, окружением и пользователем (интерфейсный подход).

Архитектура ПК

- **Альтернативный подход** основан на противопоставлении понятий «архитектура» и «организация».
- В рамках этого подхода, **архитектура** определяет возможности компьютера, а **организация** – реализацию этих возможностей в конкретных моделях.
- Правомочность такого подхода видна при сравнении моделей, принадлежащих одному семейству – как правило, эти модели обладают одной архитектурой, но разной структурной организацией.

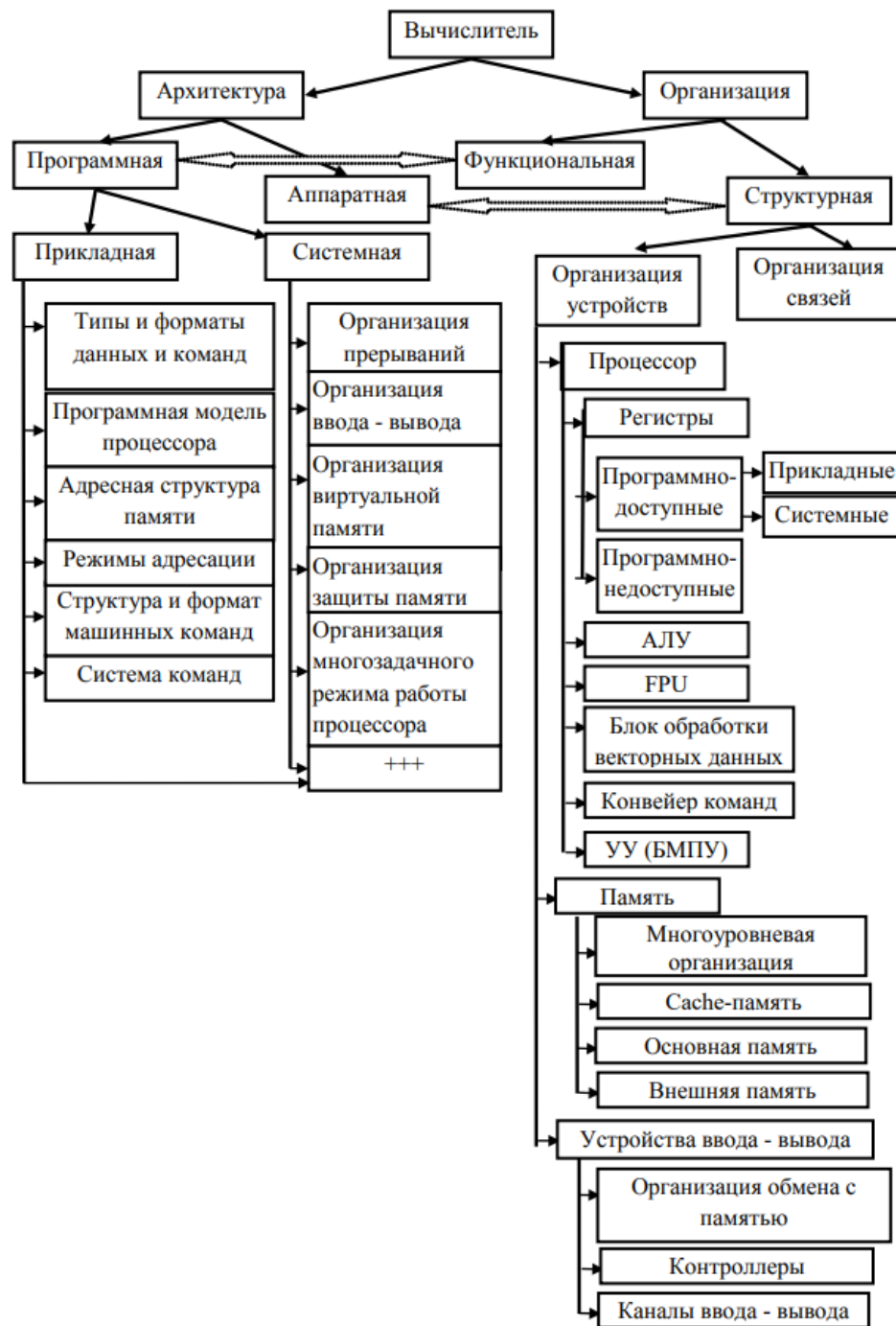
Архитектура ПК

- **При описании компьютерных систем принято различать их структурную организацию и архитектуру.** Хотя точное определение этим понятиям дать довольно трудно, среди специалистов существует общепринятое мнение о смысле этих понятий и различий между ними.
- **Термин архитектура компьютерной системы (компьютера)** относится к тем характеристикам системы, которые доступны извне, то есть со стороны программы или, с другой точки зрения, оказывает непосредственное влияние на логику выполнения программ.
- **Под термином структурная организация компьютерной системы** подразумевается совокупность операционных блоков (устройств) и их взаимосвязей, обеспечивающих реализацию спецификаций, заданных архитектурой компьютера.
- В число характеристик архитектуры входят набор машинных команд, формат разрядной сетки для представления данных разных типов, механизм обращения к средствам ввода/вывода и метод адресации памяти.
- **Характеристики структурной организации** включают скрытые от программиста детали аппаратной реализации системы: управляющие сигналы, аппаратный интерфейс между компьютером и периферийным оборудованием, технологию функционирования памяти”.

Архитектура ПК

- **Два уровня представления архитектуры компьютера:**
 - **программная архитектура** включает в себя аспекты, видимые программистам и, соответственно, программам
 - **аппаратная архитектура** включает аспекты, невидимые для программиста.
- В этом смысле понятие аппаратной архитектуры и структурной организации компьютера можно рассматривать как синонимы.
- В связи с делением программистов на прикладных и системных, программную архитектуру также можно разделить на 2 вида: прикладную и системную.

Основные элементы архитектуры и структурной организации компьютеров



Архитектура ЭВМ

- **Нет одной «единой и окончательной» номенклатуры архитектур ЭВМ.**
- Существуют: термины относятся к разным уровням – от организации памяти и исполнения до параллелизма, ISA и I/O.
- Укрупненно **можно классифицировать архитектуры ЭВМ**, следующим образом:
 1. По организации памяти и кода
 2. По модели параллелизма (классификация Флинна и далее)
 3. По способу организации процессора (микроархитектура)
 4. По типу набора инструкций (ISA)
 5. По организации данных и вычислений
 6. По памяти и иерархии хранения
 7. По вводу-выводу и шинам
 8. По организации многопроцессорности
 9. По способу адресации и доступу к памяти
 10. По безопасности и изоляции
 11. По устойчивости и надёжности
 12. Исторические и учебные архитектуры
 13. По специализированным областям применения

Архитектуры процессоров

Классификация архитектур процессоров:

- **По модели исполнения (микроархитектура)**
 - Неконвейерные (single-cycle, multi-cycle), Конвейерные (in-order), Суперскалярные in-order, Out-of-Order (OoO) суперскалярные, Спекулятивное исполнение, предсказание переходов, VLIW/EPIC, CGRA (coarse-grained reconfigurable), Асинхронные/безтактные, GALS, SMT/Hyper-Threading, CMT
- **По параллелизму и формату данных**
 - SISD, SIMD (векторные расширения), SIMT (GPU-подобная модель на процессоре/ускорителе), Матричные/систолические блоки (матрица MAC)
- **По типу ISA (логическая архитектура)**
 - RISC, CISC, VLIW/EPIC, DSP-ориентированные, Стековые/виртуальные, Capability-based
- **По организации ядер и гетерогенности**
 - Однородные многоядерные (CMP), Гетерогенные (big.LITTLE, P/E), Гибрид CPU+специализированные блоки, Многопоточные специализированные
- **По организации памяти и когерентности на уровне процессора/SoC**
 - Классическая фон-неймановская с кэшами, Модифицированная гарвардская, Scratchpad-ориентированные (DSP/RT), Когерентные NoC/CHI в многокристальных CPU
- **По энергетической организации**
 - Статические низкопотребляющие (always-on, MCU), DVFS/Power Gating/Power Islands, Энергетически пропорциональные с гетерогенностью
- **По надежности и реальному времени**
 - Lockstep/TMR ядра, Реальное время (RT)
- **По уровню интеграции и упаковке**
 - Монолитные многокристальные на кристалле (монолит), Чиплетные CPU, 2.5D/3D с HBM/Stacked SRAM
- **По назначению (доменная специализация)**
 - Общего назначения (GP), Встраиваемые/MCU/low-power, DSP/сигнальные/коммуникационные, HPC/серверные, AI/NPU/DSA
- **По модели согласования памяти (memory consistency) для программирования**
 - Сильные: TSO (x86), Слабые/релаксированные: ARM/RISC-V (с барьерами), POWER, Гетерогенные с SVM/UVM

Архитектура ЭВМ VS Архитектура процессора

• Что такое архитектура ЭВМ?

- Это совокупность принципов организации вычислительной системы целиком: модель программирования (ISA/ABI), память и её иерархия, ввод-вывод, параллелизм, адресное пространство, способы взаимодействия компонентов, требования к надежности/ безопасности.
- Иначе: что видит и чем пользуется программист и системный инженер при работе с системой в целом.

• Что такое архитектура процессора?

- Это организация и принципы построения самого процессорного ядра (или набора ядер): исполнение команд, конвейер, суперскалярность, ОоО, регистровый файл, кэш L1, механизмы предсказания ветвлений, декодер, микрокод и т. п.
- Часто отделяют ISA (логический интерфейс команд) от микроархитектуры (конкретная реализация процессора).
- В разговорной речи «архитектура процессора» иногда означает либо ISA (ARM, x86, RISC-V и т.д.), либо микроархитектуру (Zen, Golden Cove и др.).

Архитектура ЭВМ VS Архитектура процессора

- **Чем схожи**

- **Обе описывают абстракции и ограничения**, определяющие, какие программы и с какими свойствами можно выполнять.
- **Оба уровня влияют на производительность, энергоэффективность, стоимость и надежность системы.**

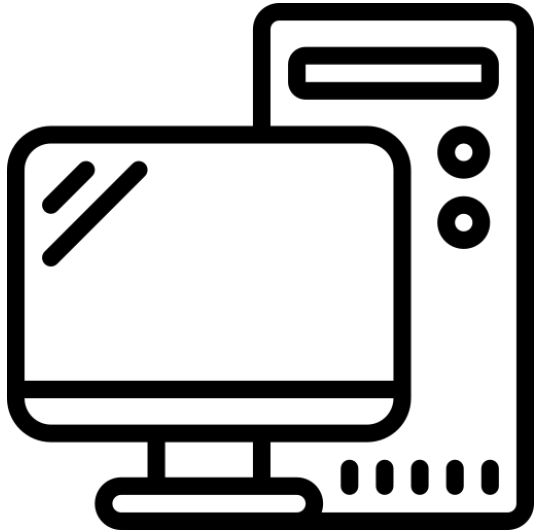
- **Чем различаются**

- **Объем:** архитектура ЭВМ шире – включает память, шины/фабрики, I/O, многопроцессорность, безопасность, виртуализацию, взаимодействие с ОС. Архитектура процессора – один компонент этой системы.
- **Видимость:** архитектура ЭВМ определяет системную модель (например, NUMA, когерентность, устройство прерываний); архитектура процессора – как отдельное ядро исполняет инструкции (pipeline, ROB, ALU и др.).
- **Инвариантность:** одну и ту же архитектуру ЭВМ можно построить на разных процессорных архитектурах, и наоборот: один ISA может быть встроен в разные ЭВМ (SMP, NUMA, SoC с разным I/O).

Архитектура ЭВМ VS Архитектура процессора

- **Архитектура ЭВМ** – системная организация всего компьютера (модель памяти, I/O, параллелизм, шины, когерентность).
- **Архитектура процессора** – организация исполнения инструкций внутри ядра(ядер) и близкой памяти, иногда также означает ISA.
- **Они связаны, но уровни разные: процессор – часть ЭВМ;** выбор архитектуры процессора определяет свойства системы, но не исчерпывает её.

Рассмотрим подробнее отдельные виды архитектур ЭВМ (классические архитектуры: архитектура фон Неймана и Гарвардская архитектура, архитектуры процессора: CISC и RISC), **укрупненно пройдемся по классификации архитектур и отдельно разберем в них некоторые архитектуры.**

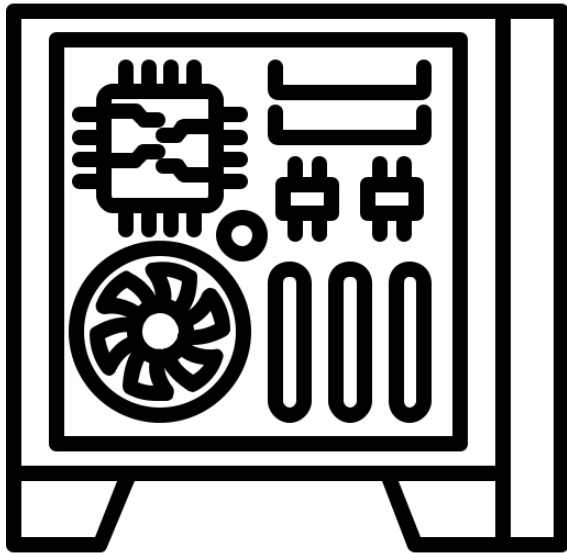


Классические архитектуры ЭВМ



Классические архитектуры ЭВМ

- **Классические архитектуры ЭВМ** — это фундаментальные модели построения вычислительных систем, заложившие основы современных компьютеров. Они определяют структуру, принципы взаимодействия компонентов и способы обработки данных.
- Наиболее значимыми считаются **архитектура фон Неймана и Гарвардская архитектура.**
- **Кроме данных архитектур, можно выделить:**
 - Архитектура с конвейеризацией (pipeline)
 - Суперскалярные и внеочередные (out-of-order) архитектуры
 - CISC (Complex Instruction Set Computing) и RISC (Reduced Instruction Set Computing) архитектуры
 - SISD, SIMD, MISD, MIMD (классификация Флинна) архитектуры
 - Архитектуры с разделяемой и распределённой памятью
 - и многие, многие другие



Архитектура Джона фон Неймана



Архитектура фон Неймана

- В 1945 г., Джон фон Нейман выделил и детально описал ключевые компоненты того, что сегодня называют «**Архитектурой фон Неймана**» современного компьютера.
- Чтобы компьютер был и эффективным, и универсальным инструментом, он должен включать следующие структуры:
 - 1) арифметико-логическое устройство (АЛУ), выполняющее арифметические и логические операции;
 - 2) устройство управления, организующее процесс выполнения программ;
 - 3) запоминающее устройство или память для хранения программ и данных;
 - 4) устройство ввода-вывода информации.

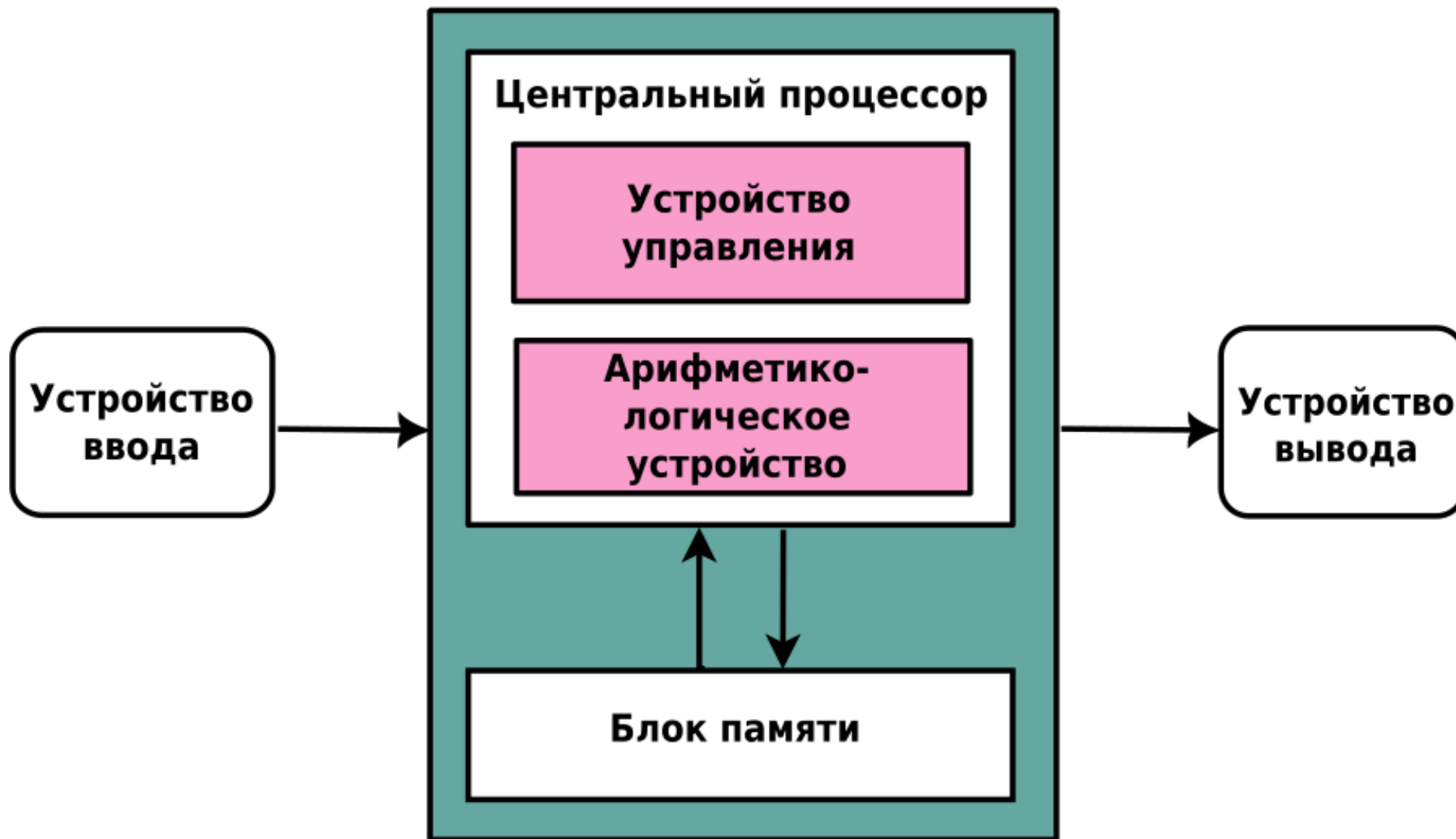


Джон фон Нейман
(1903-1957)

Архитектура фон Неймана

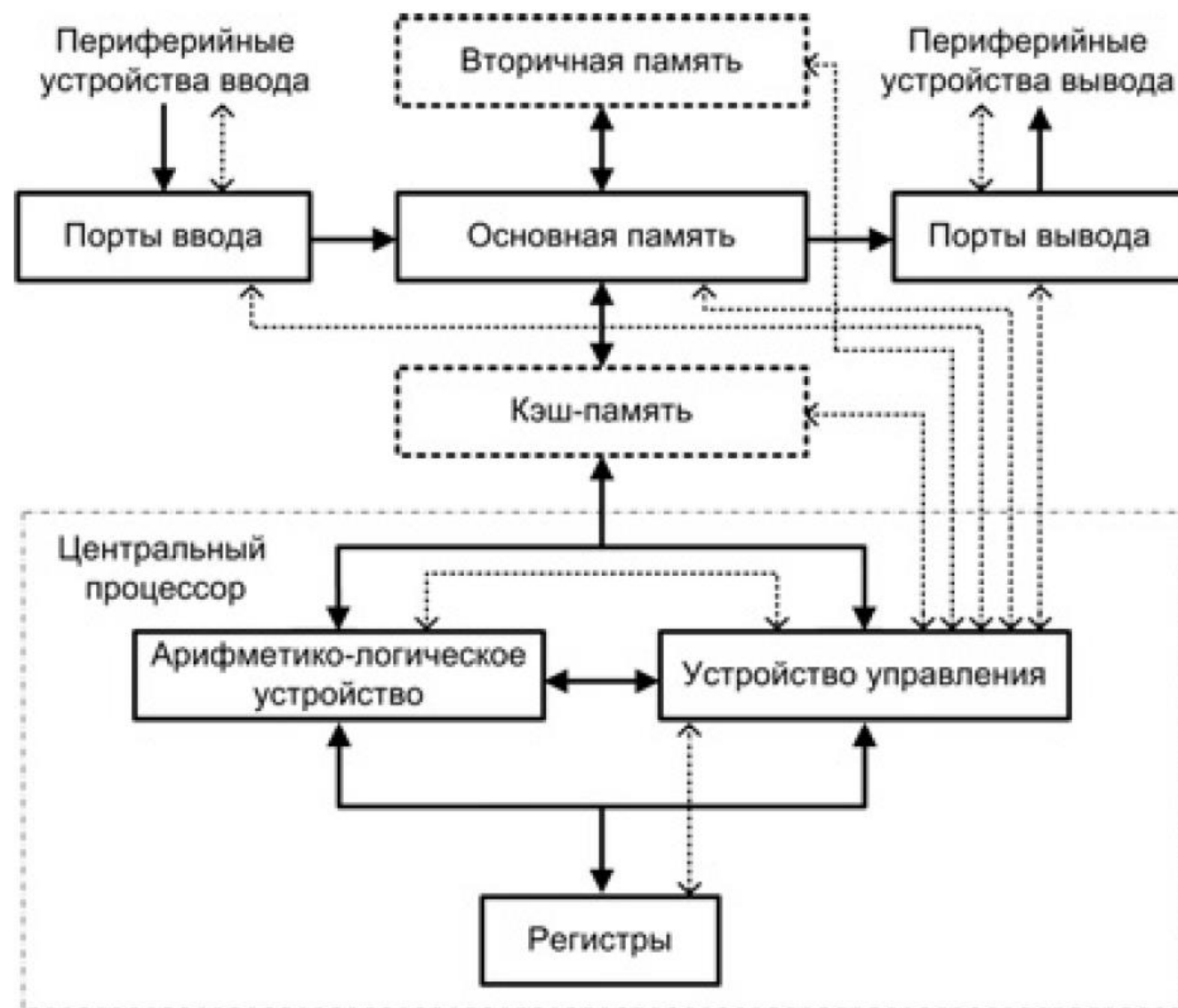
- **Архитектура фон Неймана** – это базовая модель организации современных компьютеров, где программы и данные хранятся в одной памяти, а процессор последовательно извлекает и выполняет команды.
- **Архитектура фон Неймана** – фундаментальная модель построения вычислительных систем, заложившая основы современных компьютеров. Её принципы, сформулированные в 1945 году, до сих пор лежат в основе подавляющего большинства универсальных ЭВМ.
- **Она состоит из процессора** (состоящего из устройства управления и арифметико-логического устройства), **оперативной памяти, устройств ввода/вывода и шин**, связывающих их.
- **Эта универсальная модель** обусловила широкое распространение, хотя и имеет недостаток в виде ограниченной скорости из-за **общего доступа к памяти** ("бутылочное горлышко").

Архитектура фон Неймана



Схематичное изображение архитектуры компьютера фон Неймана
(память, УУ, АЛУ, аккумулятор, ввод-вывод)

Архитектура фон Неймана



Расширенная структура фон-Неймановской вычислительной машины

Архитектура фон Неймана



Архитектура фон Неймана

Принципы фон Неймана

1. Принцип двоичного кодирования
2. Принцип программного управления
3. Принцип однородности памяти или принцип хранимой программы
4. Принцип адресности
5. Принцип иерархичности запоминающих устройств
6. Принцип параллельный организации вычислительного процесса

Архитектура фон Неймана

Принципы фон Неймана

1. Принцип двоичного кодирования.

2. Принцип программного управления работой электронно-вычислительной машины. Программы состоят из набора команд, которые выполняются процессором автоматически друг за другом в определенной последовательности.

3. Принцип однородности памяти или принцип хранимой программы. Программы и данные хранятся в одной и той же памяти. Поэтому ЭВМ не различает, что хранится в данной ячейке памяти – числа, текст или команда. Над командами можно выполнять такие же действия, как и над данными. Это позволяет создавать программы, способные в процессе вычислений изменять самих себя.

Архитектура фон Неймана

Принципы фон Неймана

4. Принцип адресности. Структурно основная память состоит из пронумерованных ячеек. Процессору в произвольный момент времени доступна любая ячейка.

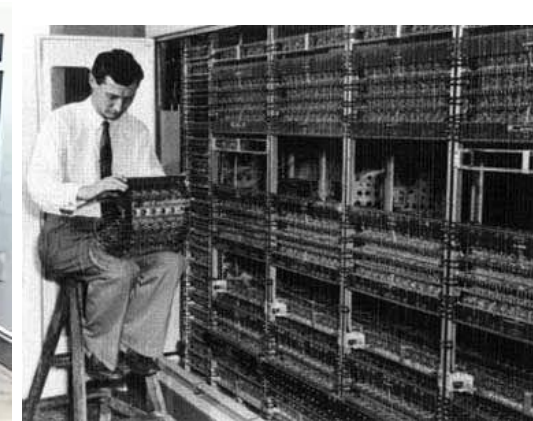
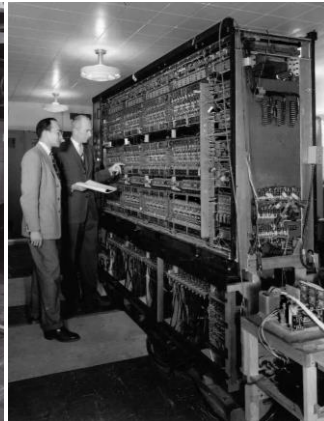
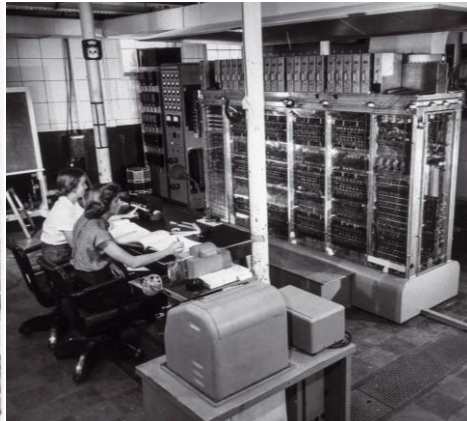
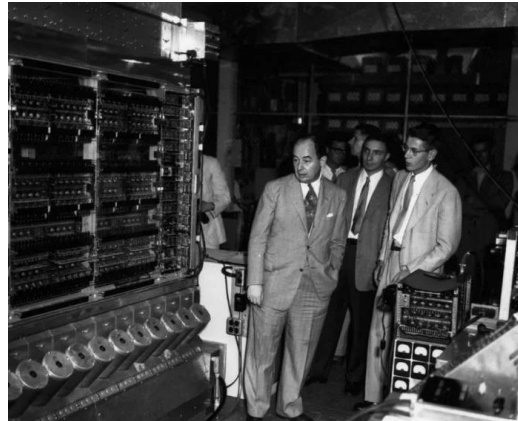
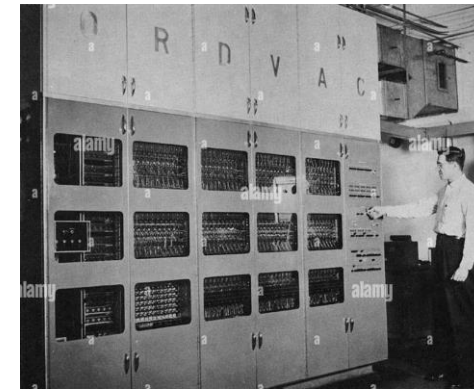
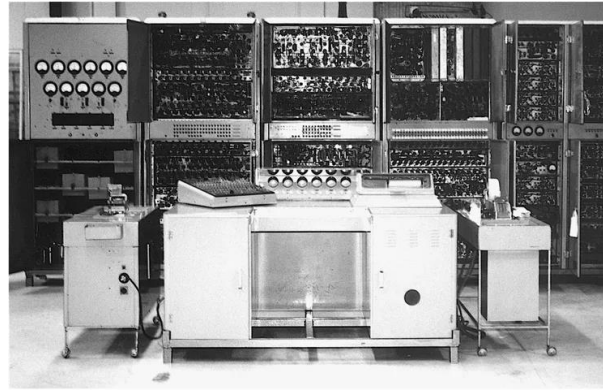
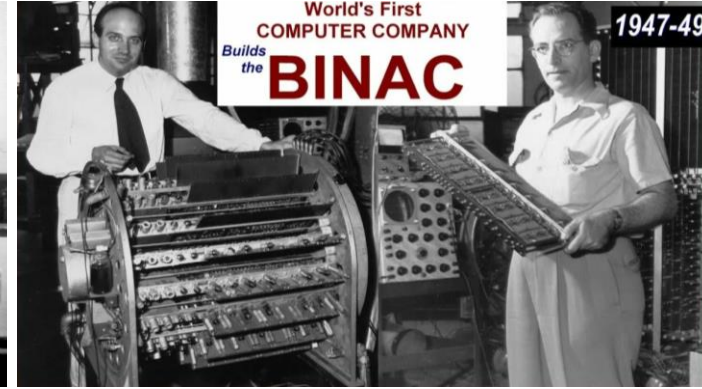
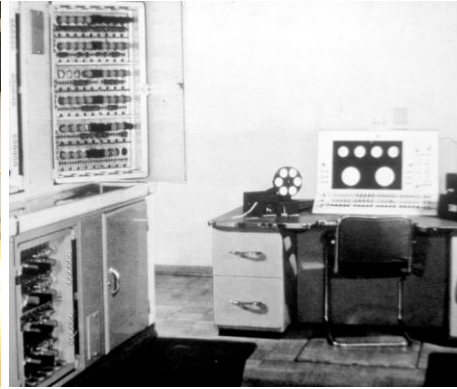
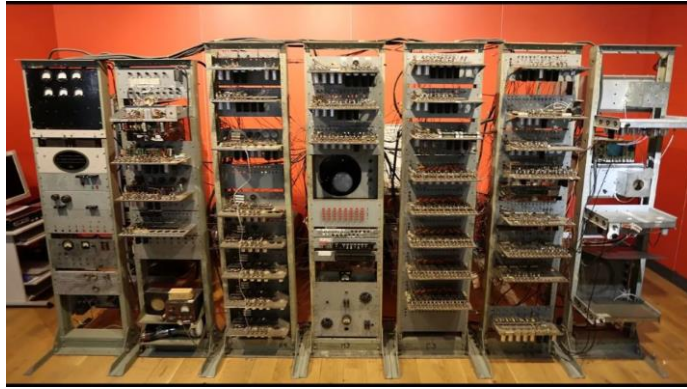
5. Принцип иерархичности запоминающих устройств. Наиболее часто используемые данные хранятся в самом быстром запоминающем устройстве сравнительно малой емкости, а более редко используемые – в самом медленном, но гораздо большей емкости.

6. Принцип параллельный организации вычислительного процесса: операции над словами производятся одновременно во всех разрядах слова.

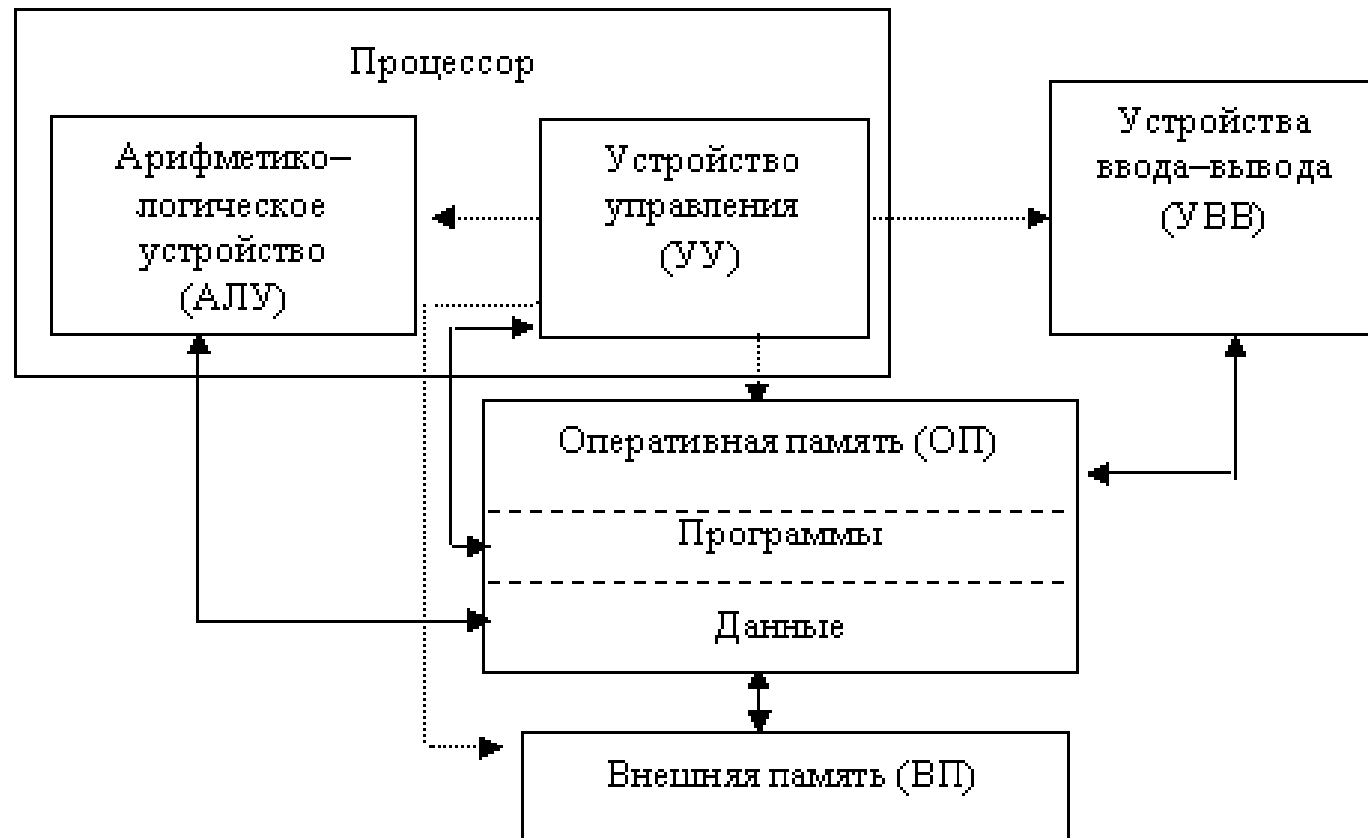
Архитектура фон Неймана

- **Первыми компьютерами, в которых были реализованы основные особенности архитектуры фон Неймана, были:**
 1. прототип — **Манчестерская малая экспериментальная машина** — Манчестерский университет, Великобритания, 21 июня 1948 года;
 2. **EDSAC** — Кембриджский университет, Великобритания, 6 мая 1949 года;
 3. **Манчестерский Марк I** — Манчестерский университет, Великобритания, 1949 год;
 4. **BINAC** — США, апрель или август 1949 года;
 5. **CSIR Mk 1** — Австралия, ноябрь 1949 года;
 6. **EDVAC** — США, август 1949 года — фактически запущен в 1952 году;
 7. **CSIRAC** — Австралия, ноябрь 1949 года;
 8. **SEAC** — США, 9 мая 1950 года;
 9. **ORDVAC** — США, ноябрь 1951 года;
 10. **IAS-машина** — США, 10 июня 1952 года;
 11. **MANIAC I** — США, март 1952 года;
 12. **AVIDAC** — США, 28 января 1953 года;
 13. **ORACLE** — США, конец 1953 года;
 14. **WEIZAC** — Израиль, 1955 год;
 15. **SILLIAC** — Австралия, 4 июля 1956 года.
- **В СССР первой полностью электронной вычислительной машиной, близкой к принципам фон Неймана, стала МЭСМ**, построенная Лебедевым (на базе киевского Института электротехники АН УССР). МЭСМ как прототип впервые была публично запущена 6 ноября 1950 года и уже в качестве полноценной машины прошла государственные приёмочные испытания 25 декабря 1951 года.

Архитектура фон Неймана



Классическая структурная схема ЭВМ



**Классическая структурная схема ЭВМ,
построенной в соответствии
с принципами фон Неймана.**

- На схеме представлены:
- **память** (оперативная и внешняя), состоящая из перенумерованных ячеек (номер ячейки называют адресом);
- **процессор**, включающий в себя устройство управления (УУ) и арифметико-логическое устройство (АЛУ); устройство ввода;
- **устройство вывода.**
- Эти устройства соединены каналами связи, по которым передается информация (управляющая – прерывистые стрелки, данные – непрерывные стрелки).

Преимущества архитектуры

- Архитектура фон Неймана, несмотря на возраст и известные ограничения, остаётся **основой подавляющего большинства современных процессоров** – от смартфонов и ноутбуков до серверов и суперкомпьютеров.
- Это не случайность, а результат сочетания её **фундаментальных преимуществ**, которые делают её **универсальной, гибкой, масштабируемой и экономически эффективной**.
- Подробно разберем ключевые преимущества и причины, по которым современные процессоры всё ещё строятся на её основе.

Преимущества архитектуры

- **1. Универсальность** (принцип хранимой программы)

«Один и тот же компьютер может решать любую задачу – достаточно изменить программу».

- Это **главное преимущество**, изменившее всю историю вычислений.
- До фон Неймана машины были жёстко запрограммированы – для смены задачи требовалось переключать провода или менять перфокарты.
- Архитектура фон Неймана ввела идею, что **программа – это данные**, которые можно хранить в памяти и менять динамически.
- **Результат:** Появление **операционных систем, интерпретаторов, компиляторов, виртуальных машин** – всего того, что делает современные компьютеры универсальными инструментами.

Преимущества архитектуры

- **2. Простота и ясность структуры**

«ЦПУ + память + шина + ввод/вывод» – легко понять, спроектировать и масштабировать.

- Архитектура предлагает **понятную и логичную модель**:
 - Процессор выбирает команду из памяти → декодирует → выполняет → переходит к следующей.
 - Все компоненты связаны через **системную шину**.
 - Память едина и адресуема.
- **Результат:** Упрощает обучение, проектирование процессоров, разработку компиляторов и отладку. Это идеальная модель для образования и базового понимания работы компьютера.

Преимущества архитектуры

- **3. Гибкость и динамическое поведение**

«Программа может менять себя в процессе выполнения».

- Поскольку код и данные хранятся в одной памяти и неотличимы на аппаратном уровне, программа может:
 - Генерировать новые команды на лету (JIT-компиляция, самоизменяющийся код).
 - Загружать модули динамически (DLL, .so).
 - Реализовывать рекурсию, полиморфизм, виртуальные функции.
- **Результат:** Поддержка сложных парадигм программирования (ООП, функциональное программирование, метапрограммирование).

Преимущества архитектуры

- **4. Экономическая и экосистемная эффективность**

«Инвестиции в ПО, ОС, инструменты, обучение – окупаются десятилетиями».

- Архитектура фон Неймана создала **гигантскую совместимую экосистему**:
 - Миллиарды строк кода, написанных под x86, ARM, RISC-V.
 - Сотни операционных систем (Windows, Linux, macOS, Android, iOS).
 - Тысячи компиляторов, отладчиков, виртуальных машин (JVM, .NET CLR).
- **Результат:** Переход на радикально новую архитектуру потребовал бы **колоссальных затрат и полной перезаписи всего ПО** – экономически нецелесообразно.

Преимущества архитектуры

• 5. Масштабируемость и адаптивность

«Можно добавлять ядра, кэши, предсказатели, SIMD – не меняя основу».

- Хотя «узкое место фон Неймана» существует, инженеры научились **обходить его**, не отказываясь от архитектуры:
 - **Кэш-память (L1, L2, L3)** – уменьшает задержки доступа к данным.
 - **Конвейеризация** – позволяет выполнять несколько команд одновременно на разных стадиях.
 - **Суперскалярность** – несколько АЛУ в одном ядре.
 - **Предсказание переходов** – уменьшает простои при ветвлениях.
 - **Многоядерность** – параллелизм на уровне ядер.
 - **SIMD-инструкции** (AVX, Neon) – параллельная обработка данных.
- **Результат:** Современные процессоры – это **гибридные системы**, где фон-неймановская основа дополняется десятками механизмов, смягчающих её недостатки.

Преимущества архитектуры

- **6. Поддержка абстракций высокого уровня**

«Архитектура позволяет строить виртуальные машины, контейнеры, облачные среды».

- Благодаря единообразию памяти и управляемому выполнению команд, **легко реализуются**:
 - **Виртуализация** (гипервизоры, VM).
 - **Контейнеризация** (Docker, Kubernetes).
 - **Управляемые среды** (Java, Python, .NET).
 - **Защита памяти** (MMU, страничная адресация).
- **Результат**: Современная ИТ-инфраструктура – от облаков до мобильных приложений – построена на этих абстракциях, которые невозможны без фон-неймановской основы.

Недостаток архитектуры

- Архитектура фон Неймана, несмотря на свою универсальность и доминирование в вычислительной индустрии, имеет **фундаментальные недостатки**, которые становятся всё более критичными по мере роста требований к производительности, энергоэффективности и безопасности.
- Эти недостатки не просто технические мелочи – они **встроены в саму концепцию архитектуры** и потому не могут быть полностью устранены без перехода к иным моделям вычислений.

Недостаток архитектуры

- **Главный недостаток: «Узкое место фон Неймана» (von Neumann Bottleneck)**
- «Процессор быстрый, а память – медленная. И они соединены одной шиной».
- Это – центральная и самая известная проблема архитектуры.
- **Суть проблемы:**
- Все команды и данные передаются между процессором и памятью **по одной и той же системной шине**. Процессор не может одновременно:
 - читать команду из памяти,
 - и читать/писать данные.
- Это создаёт **очередь на шине**, и процессор вынужден **ждать данные**, простаивая десятки или сотни тактов.
- **Последствия:**
 - **Ограничение производительности:** Даже самый быстрый CPU упирается в пропускную способность и задержки памяти.
 - **Разрыв скоростей (Memory Wall):** Современные процессоры работают на частотах 3–5 ГГц (цикл ~0.2–0.3 нс), а доступ к DRAM занимает 50–150 нс – за это время CPU мог бы выполнить сотни инструкций.
 - **Энергетические потери:** Перемещение данных между CPU и памятью потребляет больше энергии, чем сами вычисления.
- **Пример:** В задачах машинного обучения или обработки больших данных CPU тратит до 80–90% времени на ожидание данных из памяти.

Недостаток архитектуры

- Другие ключевые недостатки
- **1. Отсутствие разделения кода и данных в памяти**
- «Для процессора нет разницы между числом 42 и командой ADD».
- Это следствие принципа однородности памяти – и данные, и команды хранятся в одной памяти и представлены одинаково.
- **Проблемы:**
 - Уязвимости безопасности: Вредоносный код может перезаписать участок памяти с командами → выполнение произвольного кода (например, buffer overflow, ROP-атаки).
 - Сложность защиты: Требуются дополнительные механизмы (NX-бит, DEP, ASLR) для предотвращения исполнения данных как кода – но это «заплатки», а не решение на архитектурном уровне.
 - Ошибки программирования: Случайная запись в область памяти с кодом может привести к краху всей системы.
- **Примеры уязвимостей:** Spectre, Meltdown, Heartbleed – все они эксплуатируют особенности фон-неймановской модели.

Недостаток архитектуры

- **2. Последовательное выполнение команд**

- «Выполняй одну команду за другой – даже если они не зависят друг от друга».
- Хотя современные процессоры используют конвейеризацию, суперскалярность и спекулятивное выполнение, на логическом уровне программа всё ещё воспринимается как линейная последовательность инструкций.
- **Проблемы:**
 - Сложность параллелизма: Программист должен явно указывать, какие части кода можно выполнять параллельно (многопоточность, OpenMP, CUDA).
 - Зависимости данных: Команды часто ждут результатов предыдущих операций → простой конвейера.
 - Накладные расходы на управление потоками: Переключение контекста, синхронизация, блокировки – всё это снижает эффективность.
- **Пример:** Даже в 16-ядерном процессоре одна нитка, зависящая от результата другой, может простаивать, пока данные не придут.

Недостаток архитектуры

- **3. Сложности с многозадачностью и виртуализацией**
- «Как запустить несколько программ, чтобы они не мешали друг другу?»
- В «чистой» фон-неймановской модели нет механизмов:
 - Разделения адресного пространства.
 - Защиты памяти.
 - Прерываний и переключения контекста.
- **Что пришлось добавить «сверху»:**
 - **MMU** (Memory Management Unit) – для виртуальной памяти и защиты.
 - **Прерывания и планировщик ОС** – для переключения задач.
 - **TLB** (Translation Lookaside Buffer) – для ускорения преобразования адресов.
- Это увеличивает сложность железа и ПО, а также снижает производительность из-за накладных расходов.

Недостаток архитектуры

- **4. Низкая энергоэффективность для массовых вычислений**
- «Двигать данные – дороже, чем считать».
- В фон-неймановской архитектуре большая часть энергии тратится не на вычисления, а на:
 - Передачу данных по шине.
 - Обращения к кэшам и памяти.
 - Управление конвейером и предсказанием переходов.
- **Последствия:**
 - Для мобильных устройств и дата-центров – это критично.
 - Для задач ИИ, где нужно обрабатывать гигабайты данных – крайне неэффективно.
- **Статистика:** В GPU и AI-ускорителях до 70–90% энергии уходит на перемещение данных, а не на умножение матриц.

Недостаток архитектуры

- **5. Кризис масштабируемости**
- «Добавить ещё ядер – не значит стать быстрее».
- Закон Мура замедляется. Увеличение тактовой частоты упёрлось в **энергетическую стену**. Основной путь роста – **многоядерность**.
- **Проблемы:**
 - Все ядра делят одну память → конкуренция за шину и кэш.
 - Сложность синхронизации и согласованности кэшей (протоколы MESI и т.п.).
 - Программы не всегда можно распараллелить → закон Амдала ограничивает прирост.
- **Пример:** 64-ядерный сервер может простаивать, если приложение не умеет эффективно использовать все ядра.

Почему современные процессоры всё ещё используют архитектуру фон Неймана?

- **1. Обратная совместимость** – священная корова индустрии
- **«Если сломать совместимость – потеряешь рынок».**
 - **Пример:** Intel с 1978 года (8086) поддерживает обратную совместимость. Даже современные процессоры Core i9 могут запустить DOS-программу 1980-х (в режиме эмуляции). ARM тоже сохраняет совместимость между поколениями.
- **Последствие:** Переход на новую архитектуру = потеря всей существующей базы ПО = крах бизнеса.
- **2. Эволюция, а не революция**
- **«Не нужно изобретать велосипед – можно улучшать существующий».**
- Современные процессоры – это **не «чистые» фон-неймановские машины, а высокооптимизированные гибриды:**
 - Внутри ядра – гарвардская структура (отдельные кэши команд и данных).
 - Вне ядра – фон-неймановская шина (единая память).
 - Используются внеархитектурные методы: спекулятивное выполнение, OoO (out-of-order), SMT (hyper-threading).
- **Пример:** Процессор Apple M2 или Intel Core i9 – это фон Нейман «сверху», но внутри – сложнейшая система, обходящая его ограничения.

Почему современные процессоры всё ещё используют архитектуру фон Неймана?

- **3. Отсутствие зрелой альтернативы**
- «Dataflow, нейроморфные, квантовые – пока не готовы заменить массовые процессоры».
- Хотя активно развиваются альтернативы:
 - **Гарвардская архитектура** – используется в микроконтроллерах (AVR, PIC), но не подходит для универсальных ПК.
 - Dataflow / потоковые архитектуры – хороши для AI, но сложны в программировании.
 - **Нейроморфные чипы** (Loihi, SpiNNaker) – экспериментальны, не поддерживают ОС.
 - **Квантовые компьютеры** – не заменяют классические, а дополняют их.
- **Вывод:** Пока нет архитектуры, которая бы универсально, эффективно и совместимо заменила фон Неймана.
- **4. Программисты и инструменты «привыкли» к последовательности**
- «Большинство языков и алгоритмов написаны под последовательную модель».
- C, C++, Java, Python, JavaScript – все они предполагают **последовательное выполнение инструкций**. Переход на dataflow или параллельные модели требует **кардинального переобучения разработчиков** и переписывания методик.
- **Пример:** Даже в многопоточных программах программист думает в терминах «ПОТОКОВ», а не «ПОТОКОВ ДАННЫХ».

Будущее: фон Нейман не умрёт – он трансформируется

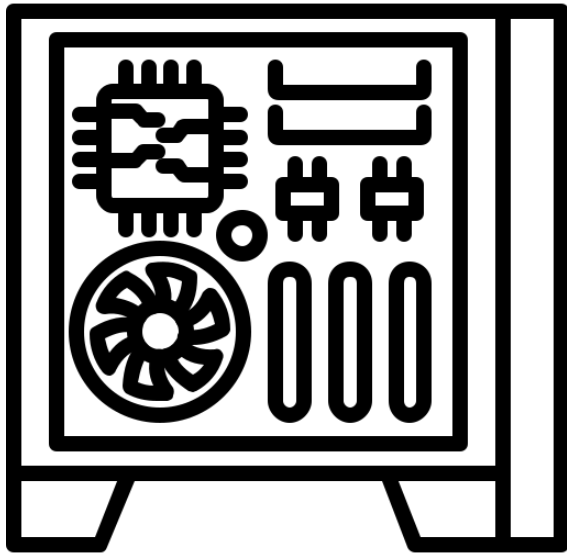
- Современные и будущие процессоры будут всё больше **отходить от «чистой» фон-неймановской модели, но сохраняя её как внешний интерфейс**:
 - **Гетерогенные системы**: CPU (фон Нейман) + GPU (dataflow) + NPU (нейроморфный) + FPGA (реконфигурируемый).
 - **Память-вычисления** (Processing-in-Memory, PIM): Вычисления прямо в памяти – частичное преодоление «узкого места».
 - **Аппаратная поддержка ИИ**: Тензорные ядра, матричные ускорители – расширяют, но не заменяют CPU.
- **Фон Нейман – это не железо, это идея.** И эта идея пока незаменима.
- **Архитектура фон Неймана – это не «устаревшая модель», а «живая основа», на которую наложены десятки слоёв оптимизаций.** Она выжила не потому, что идеальна, а потому, что достаточно хороша, гибка и поддерживает гигантскую экосистему. Её эволюция – лучший пример того, как инженеры превращают теоретические ограничения в практические решения.

Вывод

- **Архитектура фон Неймана — это фундамент, на котором построена вся современная вычислительная индустрия.** Предложенная в середине XX века, она ввела революционную идею хранимой программы, объединив код и данные в едином адресном пространстве памяти. Это позволило создать универсальные машины, способные решать любые задачи за счёт смены программного обеспечения, а не перестройки аппаратуры. Благодаря своей простоте, логичности и гибкости, архитектура быстро стала стандартом, вокруг которого выросла гигантская экосистема — от языков программирования и операционных систем до компиляторов и облачных платформ. Её принципы до сих пор лежат в основе процессоров Intel, AMD, ARM и RISC-V, что делает её, пожалуй, самой успешной и долгоживущей архитектурной моделью в истории технологий.
- **Однако за универсальность и гибкость приходится платить фундаментальными ограничениями.** Главный недостаток — «узкое место фон Неймана» — возникает из-за того, что команды и данные передаются по одной шине, вынуждая быстрый процессор простаивать в ожидании медленной памяти. К этому добавляются проблемы безопасности (отсутствие разделения кода и данных), сложности с эффективным параллелизмом, высокое энергопотребление на перемещение информации и трудности масштабирования. Эти ограничения не являются следствием устаревания технологии — они заложены в саму концепцию архитектуры и становятся всё более критичными в эпоху искусственного интеллекта, больших данных и мобильных устройств, где важны скорость, энергоэффективность и безопасность.

Вывод

- **Современные процессоры — это не «чистые» фон-неймановские машины, а сложные гибриды, которые эволюционно адаптировались к вызовам времени.** Инженеры научились обходить ограничения архитектуры: многоуровневые кэши уменьшают задержки доступа к данным, конвейеризация и спекулятивное выполнение повышают производительность, а многоядерность и SIMD-инструкции добавляют параллелизм. Внутри процессора часто используется гарвардская структура (раздельные кэши команд и данных), а снаружи — сохраняется совместимость с фон-неймановской моделью. Такой подход позволяет сохранять обратную совместимость и не переписывать триллионы строк существующего ПО, одновременно повышая эффективность систем.
- **Будущее вычислений, скорее всего, будет не в полном отказе от архитектуры фон Неймана, а в её постепенном дополнении и трансформации.** Мы уже видим, как CPU сосуществует с GPU, NPU, FPGA и другими специализированными ускорителями, каждый из которых использует свои архитектурные принципы. Появляются технологии вроде Processing-in-Memory (PIM), нейроморфных и потоковых вычислений, которые стремятся преодолеть «узкое место», интегрируя память и вычисления. Архитектура фон Неймана останется основой для общих задач, но для специализированных, высокопроизводительных или энергоэффективных приложений будут доминировать новые модели. Таким образом, фон Нейман — это не пережиток прошлого, а живая, адаптивная основа, которая продолжает развиваться, уступая место новым идеям там, где её ограничения становятся непреодолимыми.



Гарвардская архитектура

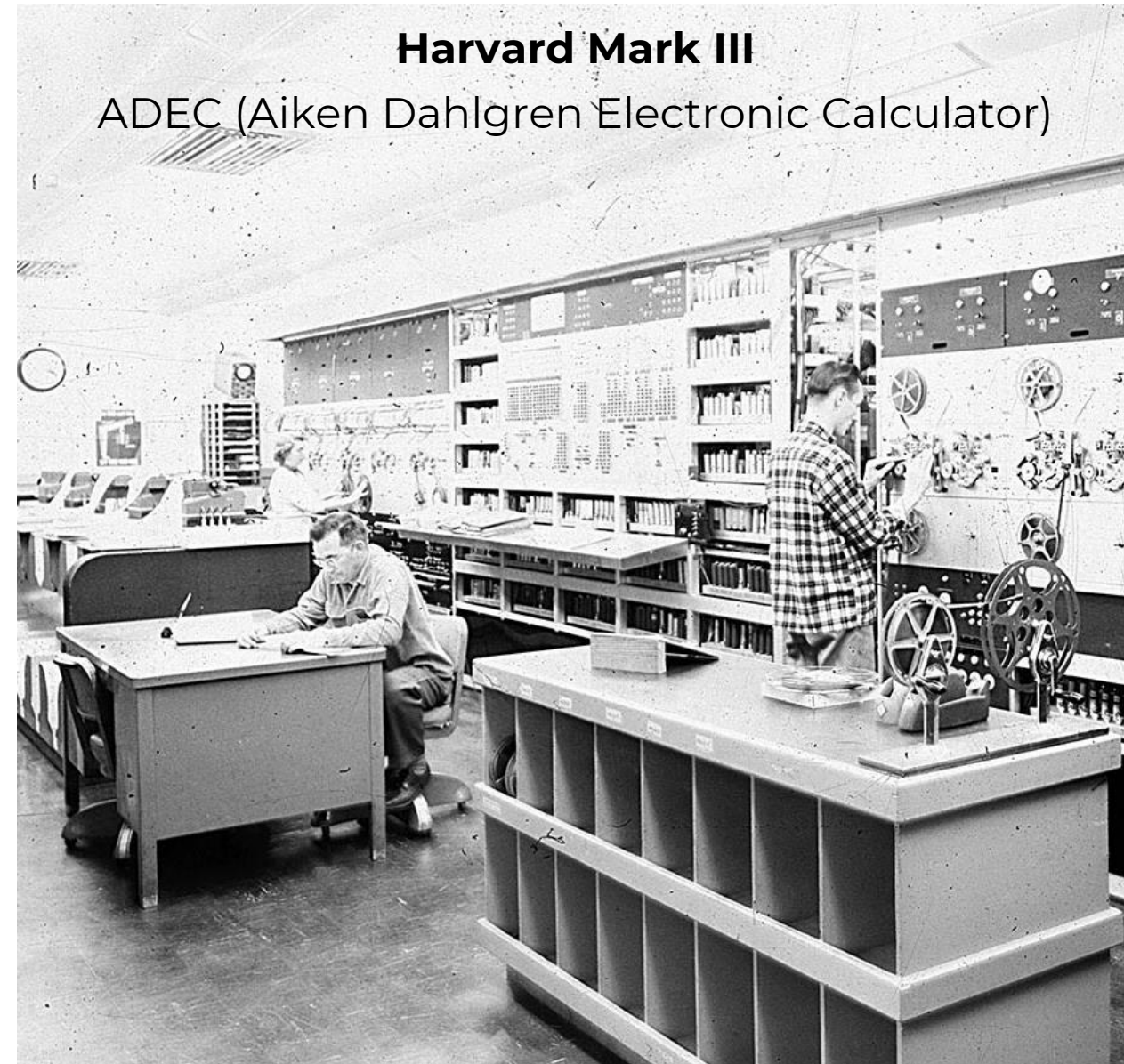


Гарвардская архитектура

- **Гарвардская архитектура** — архитектура ЭВМ, отличительными признаками которой являются:
 - хранилище инструкций и хранилище данных представляют собой разные физические устройства;
 - канал инструкций и канал данных также физически разделены.
- Архитектура была **разработана Говардом Эйкеном в конце 1930-х годов** в Гарвардском университете.
- **Достоинство Гарвардской архитектуры – параллелизм передачи, чтения и обработки команд и данных**, что повышает быстродействие, но усложняет реализацию по сравнению с фон Неймановской.
- Характеристики инструкций и данных (длина слова, временные диаграммы, структуры адресов) различны.
- **Основные отличительные признаки:**
 - **Память для инструкций и данных** – разные физические устройства;
 - **Каналы передачи инструкций и данных** – разные физические устройства;
 - **Инструкции и данные** могут представляться в разных форматах.

История возникновения

- **1944 г.: Создан компьютер Harvard Mark I** (Harvard-IBM Automatic Sequence Controlled Calculator) под руководством Говарда Эйкена.
 - **Важное уточнение:** Этот компьютер **НЕ** использовал **разделение памяти** в современном понимании! Программы задавались перфолентой, а данные — в отдельных регистрах.
 - **Название архитектуры** позже **закрепилось за принципом разделения памяти**, который фактически не был реализован в Mark I.
- **1947 г.:** Проект **Harvard Mark III** уже включал разделённую память для команд и данных — это **считается первым применением "настоящей" Гарвардской архитектуры**.
- **1970-е гг.:** Принцип стал массово использоваться в **микроконтроллерах** (например, в ранних чипах Intel MCS-51).
- **Термин «Гарвардская архитектура» появился позже как обобщение принципов**, а не как описание конкретной машины Mark I.
- Гарвардская архитектура используется в ПЛК и микроконтроллерах, таких, как Microchip PIC, Atmel AVR, Intel 4004, Intel 8051, а также в кэш-памяти первого уровня x86-микропроцессоров, делящейся на два равных либо различных по объёму блока для данных и команд.



Гарвардская архитектура

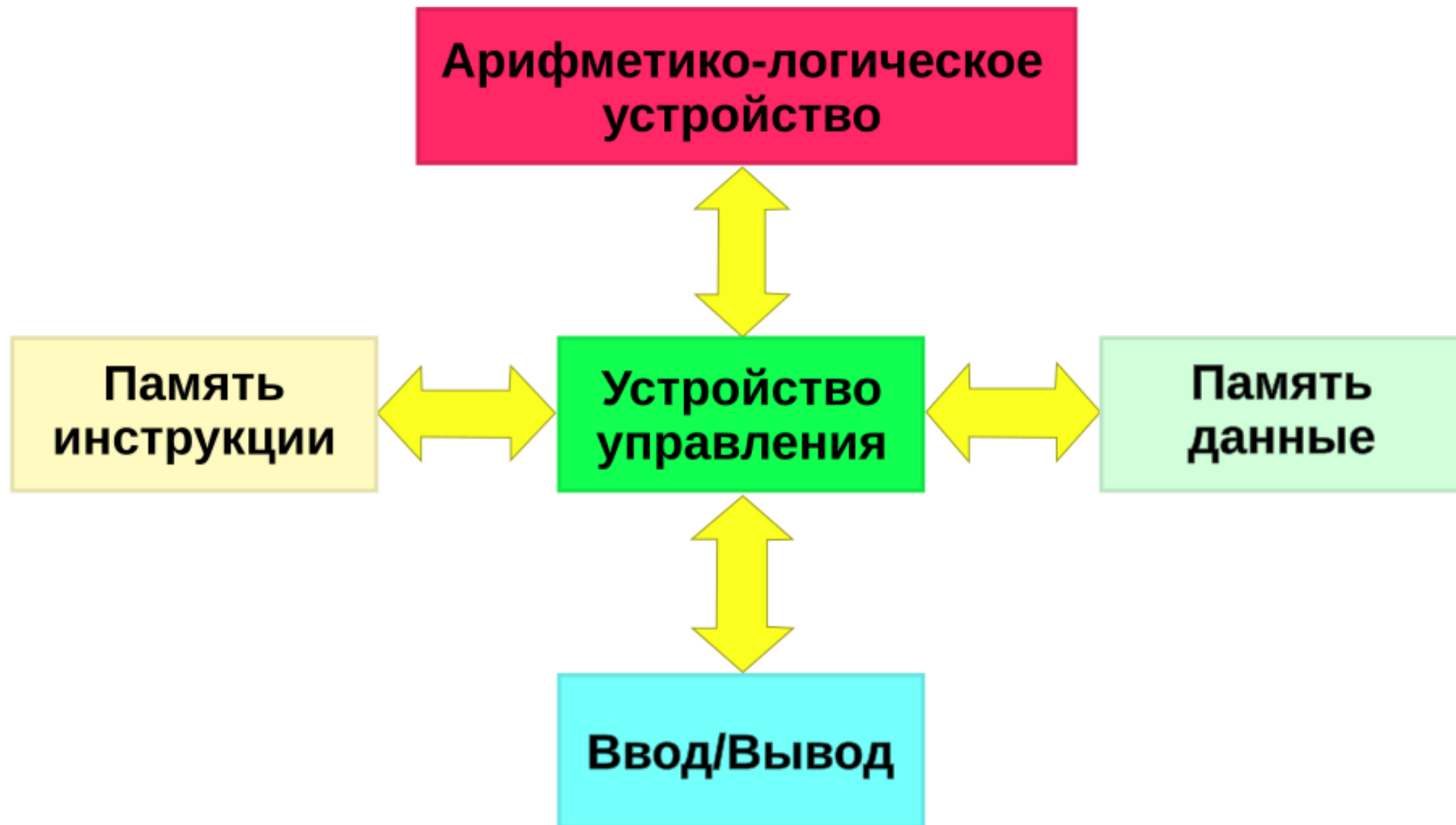
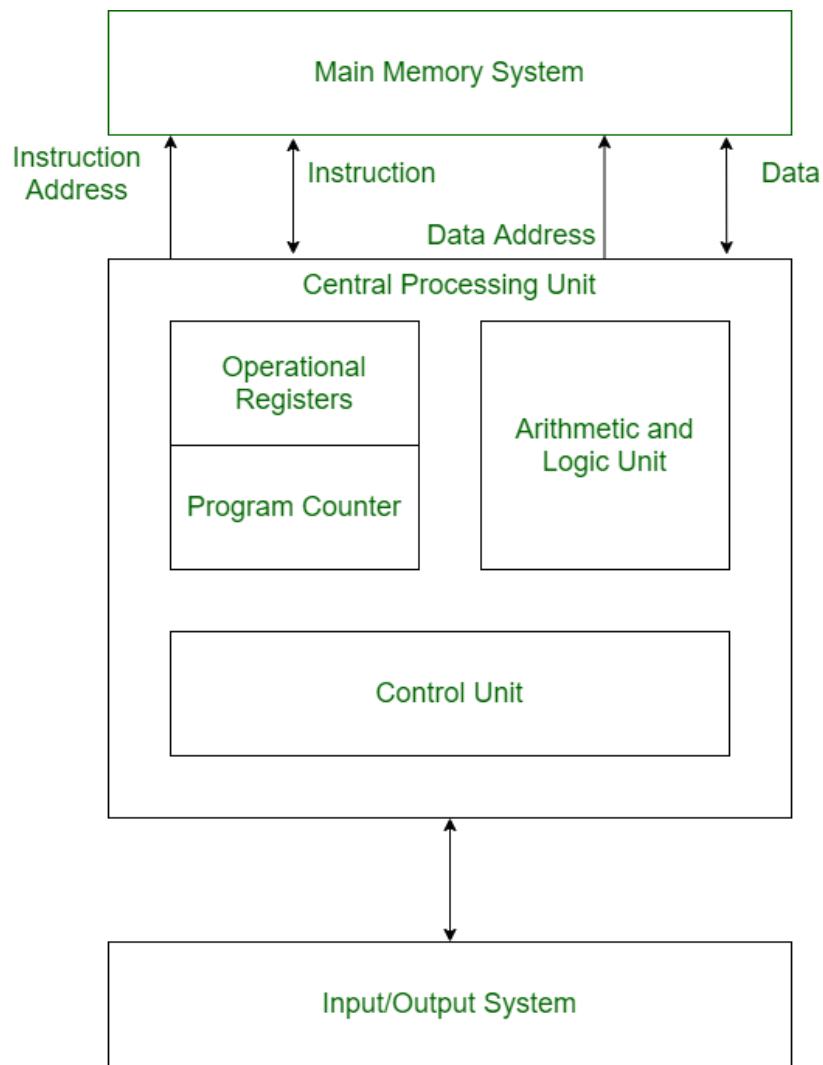


Схема Гарвардской архитектуры компьютера

Гарвардская архитектура



Гарвардская архитектура — модель ЭВМ, в которой программа и данные хранятся в физически разделённых областях памяти с независимыми шинами для их доступа.

Это позволяет:

- Одновременно читать команду и обрабатывать данные,
- Полностью изолировать программный код от данных.

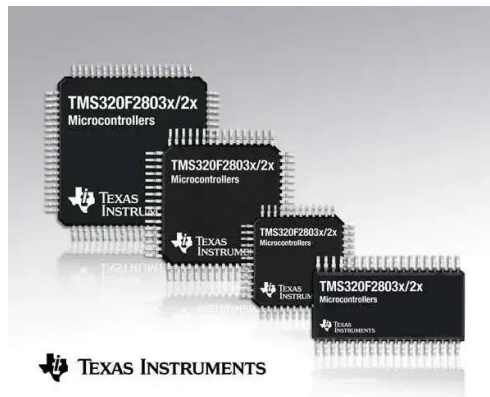
Примеры применения Гарвардской архитектуры

Микроконтроллеры



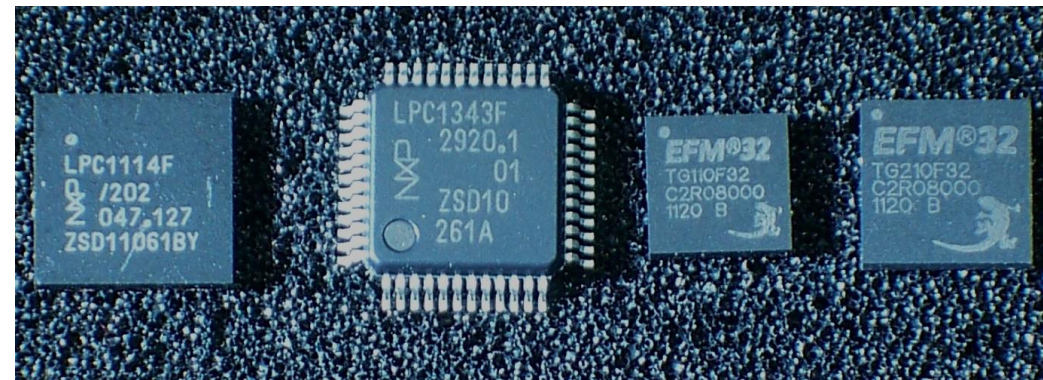
AVR (Atmel/Microchip)

DSP-процессоры



Texas Instruments TMS320

Современные процессоры



ARM Cortex-M (Cortex-M0/M3)



PIC (Microchip) PIC16F877A



Analog Devices Blackfin
ADSP-BF533



RISC-V с расширением Zfinx

Чистая Гарвардская архитектура

- Чистая Гарвардская архитектура (Pure Harvard Architecture) представляет собой теоретическую модель, где **память программ и память данных являются абсолютно разными физическими устройствами**, например, ПЗУ и ОЗУ соответственно.
- Между этими устройствами полностью отсутствуют какие-либо пути для записи в память программ, что обеспечивает максимальную защиту кода.
- Системная шина полностью разделена: существуют отдельная шина команд и отдельная шина данных.
- Это позволяет процессору одновременно осуществлять доступ к инструкции и к данным, что обеспечивает максимальную производительность.
- Однако этот подход имеет существенный недостаток: большое количество выводов на кристалле процессора, что увеличивает его размер, сложность и стоимость производства.
- Из-за этих факторов чистая Гарвардская архитектура применяется редко и в основном в очень специфических и критически важных системах, где производительность и надежность превалируют над стоимостью.

Модифицированная гарвардская архитектура

- Соответствующая схема реализации доступа к памяти имеет один очевидный **недостаток — высокую стоимость**.
- При разделении каналов передачи команд и данных на кристалле процессора последний должен иметь почти вдвое больше интерфейсных выводов, так как шина адреса и шина данных составляют основную часть выводов микропроцессора.
- Способом разрешения этой проблемы стала идея **использовать общие шину данных и шину адреса для всех внешних данных, а внутри процессора использовать шину данных, шину команд и две шины адреса**.
- Такую концепцию стали называть **модифицированной гарвардской архитектурой**.
- Такая схемотехника **применяется в современных сигнальных процессорах**. Ещё дальше по пути уменьшения стоимости пошли при создании однокристалльных микроЭВМ — **микроконтроллеров**. В них одна шина команд и данных применяется и внутри кристалла.
- Разделение шин в модифицированной гарвардской структуре осуществляется при помощи отдельных управляющих сигналов: чтения, записи или выбора области памяти.

Гибридные модификации с архитектурой фон Неймана

- Существуют гибридные архитектуры, сочетающие достоинства как гарвардской, так и фон-неймановской архитектур.
- **Современные CISC-процессоры** обладают отдельной кэш-памятью 1-го уровня для команд и данных, что позволяет им за один рабочий такт получать одновременно и команду, и данные для её выполнения.
- **То есть процессорное ядро аппаратно гарвардское, но программно оно фон-неймановское, что упрощает написание программ.**
- Обычно в данных процессорах одна шина используется и для передачи команд, и для передачи данных, что схемотехнически упрощает систему.
- Современные варианты таких процессоров могут иногда содержать встроенные контроллеры сразу нескольких разнотипных шин для работы с различными типами памяти — например, DDR RAM и Flash.
- Тем не менее, и в этом случае шины, как правило, используются и для передачи команд, и для передачи данных без разделения, что делает данные процессоры ещё более близкими к фон-неймановской архитектуре при сохранении достоинств гарвардской архитектуры.

Преимущества архитектуры

- **Преимущества** Гарвардской архитектуры обусловлены её ключевым принципом – **физическим разделением памяти и шин для команд (инструкций) и данных**. Это разделение открывает путь к ряду важных технических и эксплуатационных выгод, особенно в специализированных системах.
- **1. Повышенная производительность (параллелизм доступа к памяти)**
- **Одновременный доступ к командам и данным:** Процессор может в один и тот же такт выбирать следующую инструкцию из памяти программ и одновременно читать/писать данные из/в память данных. Это устраняет «узкое горлышко фон Неймана», где доступ к командам и данным происходит поочерёдно через одну шину.
- **Сокращение простоев процессора:** Отсутствие ожидания завершения операции с памятью для выполнения следующего шага увеличивает эффективность конвейера (pipeline).
- **Особенно важно для DSP и MCU:** В цифровых сигнальных процессорах (DSP) и микроконтроллерах (MCU), где критична скорость обработки потоков данных (например, аудио, видео, управление в реальном времени), это преимущество становится решающим.
- **Пример:** В DSP с расширенной Гарвардской архитектурой (SHARC) за один такт можно загрузить команду, два операнда и записать результат – идеально для алгоритмов типа БПФ или фильтрации.

Преимущества архитектуры

- **2. Повышенная надёжность и безопасность**
- **Защита кода от модификации:** Память программ часто реализуется как ПЗУ (ROM, Flash), физически отделённое от ОЗУ данных. Это исключает случайное или злонамеренное изменение исполняемого кода во время работы.
- **Устойчивость к сбоям:** В системах реального времени (автомобили, авионика, медицинские устройства) такая защита критически важна – сбой в данных не может повредить саму программу.
- **Снижение риска эксплойтов:** Невозможность выполнения кода из области данных (DEP – Data Execution Prevention) реализуется на аппаратном уровне, что затрудняет атаки типа buffer overflow.
- **Контраст с фон Нейманом:** В фон-неймановской архитектуре код и данные в одной памяти – переполнение буфера может привести к перезаписи кода и выполнению вредоносных инструкций.

Преимущества архитектуры

- **3. Оптимизация под специализированные задачи**
- **Раздельная оптимизация памяти:** Память программ может быть медленной, но энергоэффективной (Flash), а память данных – быстрой (SRAM). Это позволяет гибко подбирать типы памяти под задачи.
- **Разные адресные пространства:** Программа и данные могут иметь независимые адресные пространства, что упрощает проектирование и позволяет эффективнее использовать ограниченные ресурсы (например, в 8-битных MCU).
- **Поддержка реального времени (RTOS):** Предсказуемое время доступа к памяти команд и данных упрощает планирование задач в системах с жёсткими временными ограничениями.

Преимущества архитектуры

- **4. Энергоэффективность (в некоторых реализациях)**
- **Целенаправленное управление питанием:** Раздельные шины и контроллеры памяти позволяют отключать неиспользуемые блоки (например, шину данных, если выполняется только выборка команд).
- **Меньше конфликтов шины:** Отсутствие арбитража за общую шину снижает энергопотребление на уровне сигнализации.
- **Актуально для IoT и мобильных устройств:** В устройствах с батарейным питанием экономия энергии критична – Гарвардская архитектура помогает продлить время автономной работы.

Преимущества архитектуры

- **5. Упрощение аппаратной реализации в микроконтроллерах**
- **Модифицированная Гарвардская архитектура** (Modified Harvard) – наиболее распространённый вариант – сочетает внутреннее разделение шин (для производительности) с внешней общей шиной (для экономии выводов и стоимости).
- **Меньше конфликтов при доступе к памяти:** Внутри чипа нет необходимости в сложных схемах арбитража шины, что упрощает дизайн и повышает частоту работы.
- **Идеальна для встраиваемых систем:** Низкая стоимость, малый размер кристалла, высокая предсказуемость поведения – всё это делает её выбором №1 для MCU (PIC, AVR, 8051, ARM Cortex-M).

Преимущества архитектуры

- **6. Совместимость с современными гибридными архитектурами**
- **Гарвардские принципы используются даже в фон-неймановских процессорах:** Современные CPU (x86, ARM Cortex-A) используют отдельные кэши L1 для инструкций и данных – это фактически Гарвардская архитектура на уровне кэша.
- **Лучшее из двух миров:** Программист видит единое адресное пространство (фон Нейман), но аппаратно система работает как Гарвардская – обеспечивая и гибкость, и производительность.
- **Эволюционное преимущество:** Гарвардская архитектура не вытеснена – она интегрирована и адаптирована, став неотъемлемой частью современных процессоров.

Когда Гарвардская архитектура особенно выгодна?

- **В встраиваемых системах** (микроконтроллеры, датчики, бытовая техника).
- **В цифровой обработке сигналов** (DSP – аудио, видео, связь).
- **В системах реального времени** (автомобили, промышленная автоматика, авионика).
- **В устройствах с ограниченным питанием** (IoT, носимая электроника).
- **В критически важных системах**, где отказ недопустим (медицинские приборы, военная техника).

Недостатки архитектуры

- Несмотря на значительные преимущества, Гарвардская архитектура имеет и существенные недостатки, которые ограничивают её применение в универсальных вычислительных системах и создают определённые сложности при разработке программного обеспечения и аппаратного дизайна.
- Эти недостатки особенно заметны при сравнении с фон-неймановской архитектурой.

Недостатки архитектуры

- **1. Сложность и высокая стоимость аппаратной реализации**
- **Удвоение шин и контроллеров:** Требуются отдельные шины команд и данных, отдельные контроллеры памяти, буферы и дешифраторы адресов → увеличение площади кристалла и числа выводов (pins).
- **Дороже в производстве:** Особенно в ранние годы электроники (1940–1970-е), когда каждый транзистор и вывод имели высокую стоимость. Это стало одной из главных причин доминирования фон-неймановской архитектуры в массовых компьютерах.
- **Проблема с внешними устройствами:** В чистой Гарвардской архитектуре внешние устройства (например, загрузчик) должны иметь отдельные интерфейсы для записи в память программ и данных → усложнение системной платы.
- **Решение:** Модифицированная Гарвардская архитектура использует общую внешнюю шину – это снижает стоимость, но частично жертвует преимуществами чистой реализации.

Недостатки архитектуры

- **2. Неэффективное использование памяти**
- **Жёсткое разделение объёмов памяти:** Память программ и память данных физически разделены – нельзя динамически перераспределить свободное место из одной области в другую.
 - **Пример:** если в программе мало кода, но много данных, свободное место в памяти программ остаётся неиспользованным.
- **Фрагментация ресурсов:** Разработчик вынужден заранее оценивать объёмы кода и данных, что может привести к неоптимальному использованию чипа.
- **Ограничение гибкости:** В фон-неймановской архитектуре можно динамически выделять память под стек, кучу, код (например, JIT-компиляция), что невозможно в чистой Гарвардской системе.
- **Современные MCU частично решают это:** например, в некоторых ARM Cortex-M есть возможность выполнять код из RAM (XIP – Execute In Place), но это требует дополнительных механизмов и снижает безопасность.

Недостатки архитектуры

- **3. Сложность программирования и отладки**

- **Раздельные адресные пространства:** Программист должен явно указывать, к какой памяти он обращается – к памяти программ (например, для чтения констант) или к памяти данных.
 - В языках низкого уровня (ассемблер, C с расширениями) это требует использования специальных директив или функций (например, `PROGMEM` в AVR-GCC).
- **Ограниченная поддержка динамических структур:** Невозможно легко реализовать самомодифицирующийся код, JIT-компиляторы, динамическую загрузку модулей – всё это естественно для фон-неймановской архитектуры.
- **Сложности с отладкой:** Отладчики должны уметь работать с двумя разными пространствами памяти, что усложняет реализацию симуляторов и эмуляторов.
- **Пример:** В микроконтроллерах AVR для чтения строки из Flash-памяти нужно использовать специальную функцию `pgm_read_byte()`, а не обычный указатель – это создаёт дополнительную нагрузку на разработчика.

Недостатки архитектуры

- **4. Ограниченная гибкость архитектуры**

- **Невозможность динамической генерации кода:** Такие технологии, как JIT (Just-In-Time компиляция), самоизменяющийся код, динамические плагины – невозможны или сильно затруднены.
- **Проблемы с современными ОС и виртуализацией:** Большинство операционных систем предполагают единое адресное пространство и возможность выполнения кода из любой области памяти – это несовместимо с чистой Гарвардской моделью.
- **Трудности с поддержкой сложных языков:** Языки с интенсивным использованием метапрограммирования, reflection, динамической компиляции (например, Python, JavaScript, Java в некоторых режимах) работают неэффективно или требуют эмуляции.

Недостатки архитектуры

- **5. Компромиссы в модифицированных реализациях**
- **Потеря преимуществ при использовании общей внешней шины:** В модифицированной Гарвардской архитектуре внутренние шины разделены, но внешняя – общая. При частом обращении к внешней памяти возникает «узкое горлышко», как в фон-неймановской системе.
- **Зависимость от кэшей:** В гибридных системах (например, ARM/x86 с отдельными L1-кэшами) производительность сильно зависит от эффективности кэширования. При промахах кэша система деградирует до фон-неймановского поведения.
- **Сложность синхронизации:** Если код может обновляться (например, через загрузчик), требуется механизм синхронизации между памятью программ и кэшем инструкций – иначе процессор может выполнять устаревший код.

Недостатки архитектуры

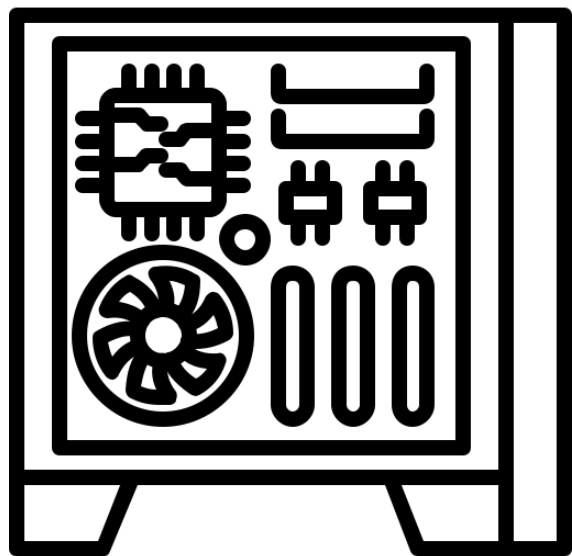
- **6. Ограниченная применимость в универсальных системах**
- **Не подходит для ПК, серверов, рабочих станций:** Современные ОС, языки программирования, среды выполнения (runtime) и приложения требуют гибкости фон-неймановской модели.
- **Невозможность реализации некоторых алгоритмов:** Например, генетические алгоритмы, эволюционное программирование, где код модифицируется в процессе выполнения – несовместимы с жёсткой изоляцией кода.
- **Меньше поддержки со стороны инструментов:** Компиляторы, линковщики, отладчики для фон-неймановских систем более развиты и стандартизированы.

Выводы

- **Гарвардская архитектура** представляет собой фундаментальный подход к организации вычислительных систем, **основанный на физическом разделении памяти и шин для команд и данных**. Её **ключевые преимущества** – **повышенная производительность** за счёт параллельного доступа к инструкциям и операндам, а также **повышенная надёжность** и безопасность благодаря аппаратной изоляции кода от данных. Эти особенности делают её идеальной для систем, где критичны предсказуемость, скорость реакции и устойчивость к сбоям. Однако за эти преимущества приходится платить: **Гарвардская архитектура сложнее в реализации, дороже в производстве**, менее гибка в программировании и неэффективно использует память. Особенно остро эти недостатки проявляются в универсальных вычислительных системах, где требуется динамическое управление памятью, поддержка сложных языков и операционных систем.
- Современное развитие показывает, что чистая Гарвардская архитектура практически не используется – вместо неё доминируют **модифицированные и гибридные реализации**. **Микроконтроллеры применяют модифицированную версию** (внутреннее разделение шин, внешняя – общая), что снижает стоимость и сохраняет производительность. Универсальные процессоры (x86, ARM Cortex-A) используют гибридный подход: на уровне программиста – фон-неймановская модель, на аппаратном уровне – отдельные кэши L1 для команд и данных. Это позволяет сочетать гибкость универсальных систем с производительностью, характерной для Гарвардской архитектуры. Таким образом, принципы Гарвардской архитектуры не исчезли – они эволюционировали и стали неотъемлемой частью современного процессорного дизайна.

Область применения

- **Гарвардская архитектура** (в чистом, модифицированном или гибридном виде) находит широкое применение там, где важны **производительность, энергоэффективность, надёжность и предсказуемость**:
 - **Микроконтроллеры (MCU)** – основа встраиваемых систем: PIC, AVR, 8051, ARM Cortex-M. Используются в бытовой технике, автомобильной электронике, промышленных контроллерах, IoT-устройствах.
 - **Цифровые сигнальные процессоры (DSP)** – обработка аудио, видео, радиосигналов в реальном времени: семейства Texas Instruments C6000, Analog Devices SHARC, Freescale MSC.
 - **Системы реального времени (RTOS)** – авионика, медицинское оборудование, АСУ ТП, робототехника — где сбой недопустим, а реакция должна быть мгновенной.
 - **Безопасные и критически важные системы** – военная техника, системы управления АЭС, железнодорожная автоматика – где изоляция кода предотвращает катастрофические последствия.
 - **Устройства с ограниченным питанием** – носимая электроника, сенсоры, беспроводные узлы IoT – где энергоэффективность и автономность важнее гибкости.
- Таким образом, Гарвардская архитектура остаётся одной из самых востребованных в мире цифровой электроники – не как конкурент фон-неймановской модели, а как её мощное **дополнение, оптимизированное под специализированные, надёжные и высокопроизводительные задачи**.



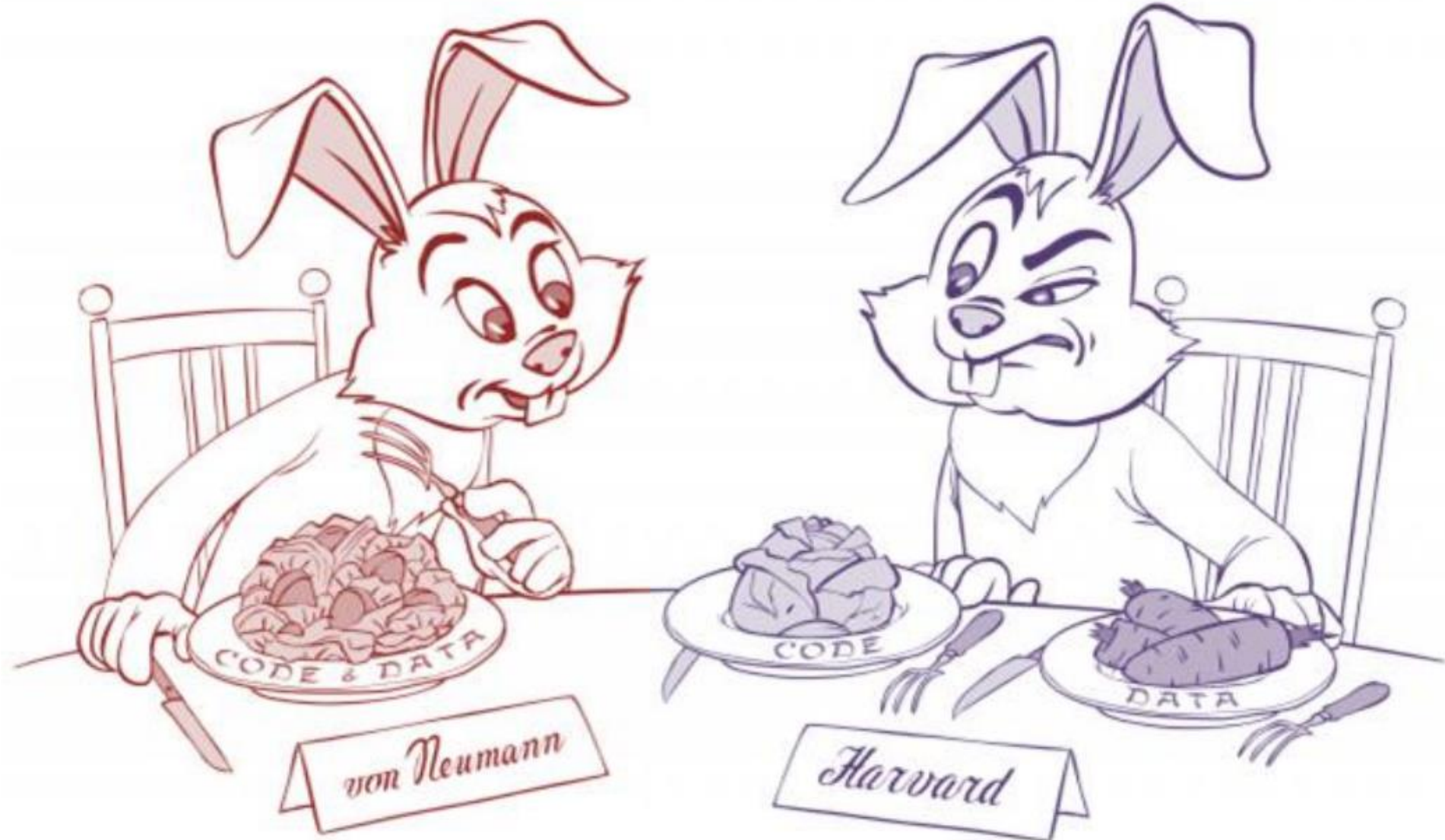
Архитектура фон Неймана vs Гарвардская архитектура



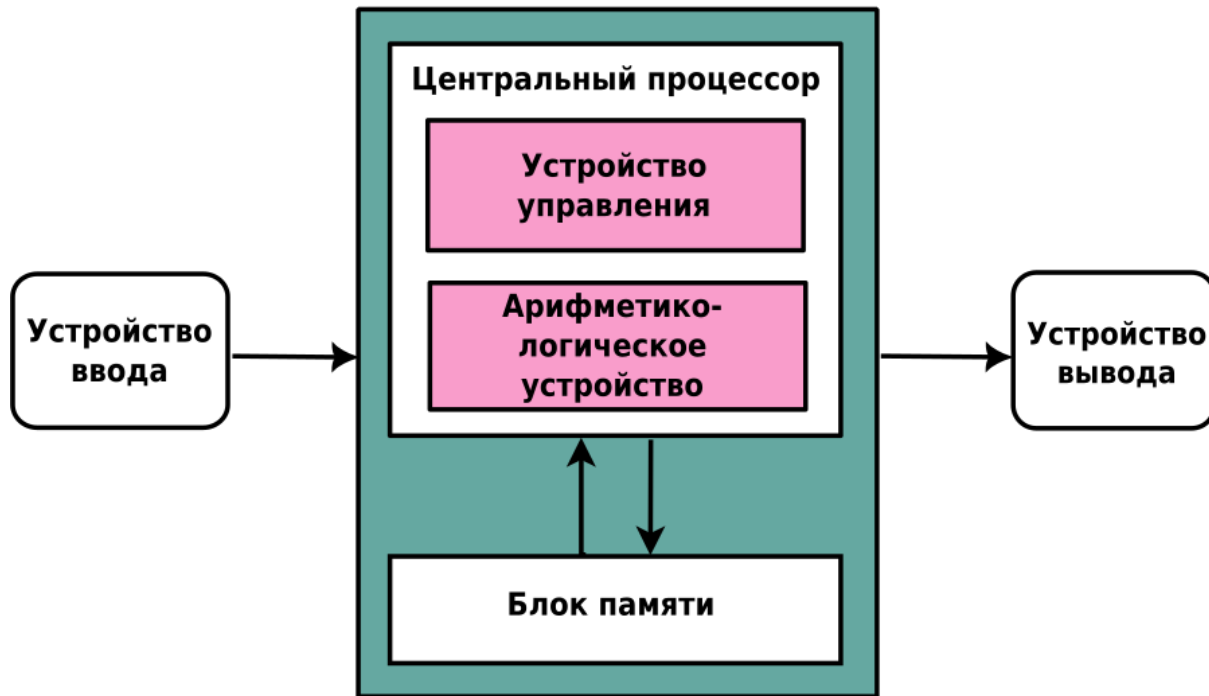
Классические архитектуры ЭВМ

- **Гарвардская архитектура** предполагает **раздельное использование памяти программ и данных**. Обычно такую архитектуру используют для повышения быстродействия системы за счет разделения путей доступа к памяти программ и данных. Большинство специализированных микропроцессоров (особенно **микроконтроллеры**) имеют данную архитектуру.
- Антипод гарвардской – **архитектура фон Неймана** – **предполагает хранение программ и данных в общей памяти** и наиболее характерна для микропроцессоров, ориентированных на использование в компьютерах. Примером могут служить **микропроцессоры семейства x86**.

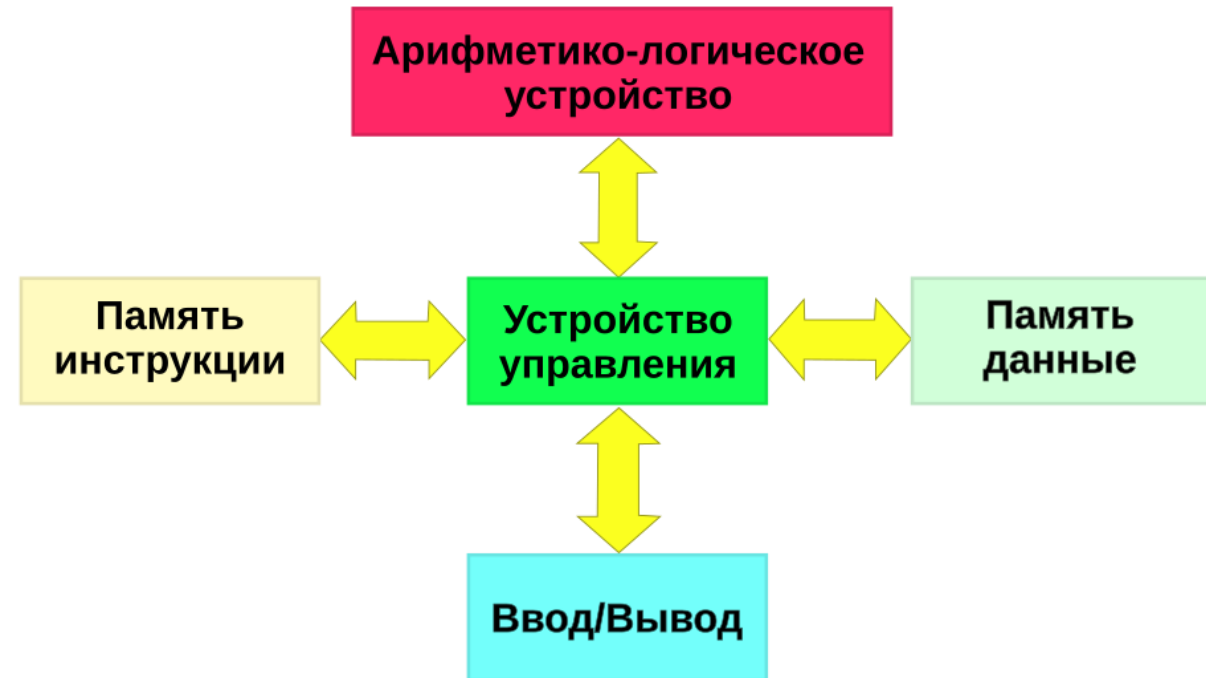
Архитектура фон Неймана vs Гарвардская архитектура



Архитектура фон Неймана vs Гарвардская архитектура

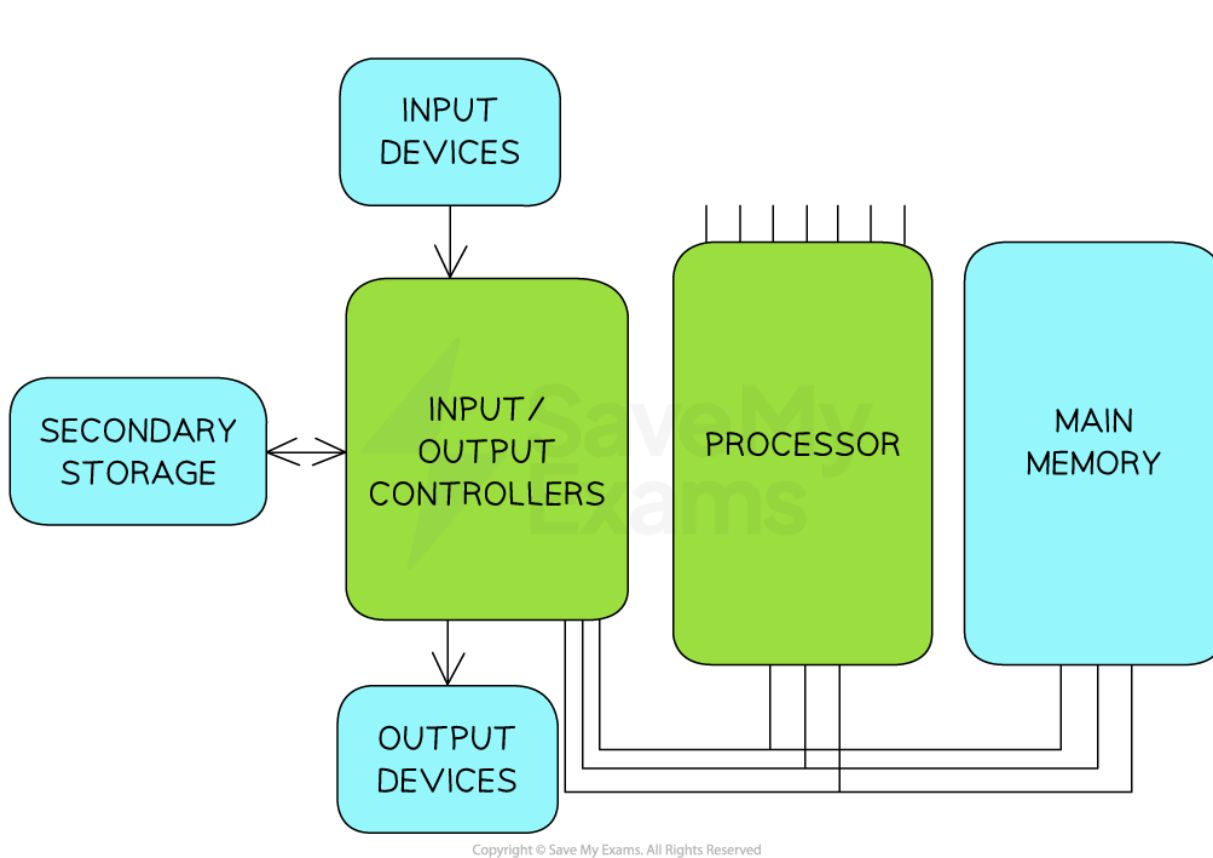


Схематичное изображение архитектуры фон Неймана

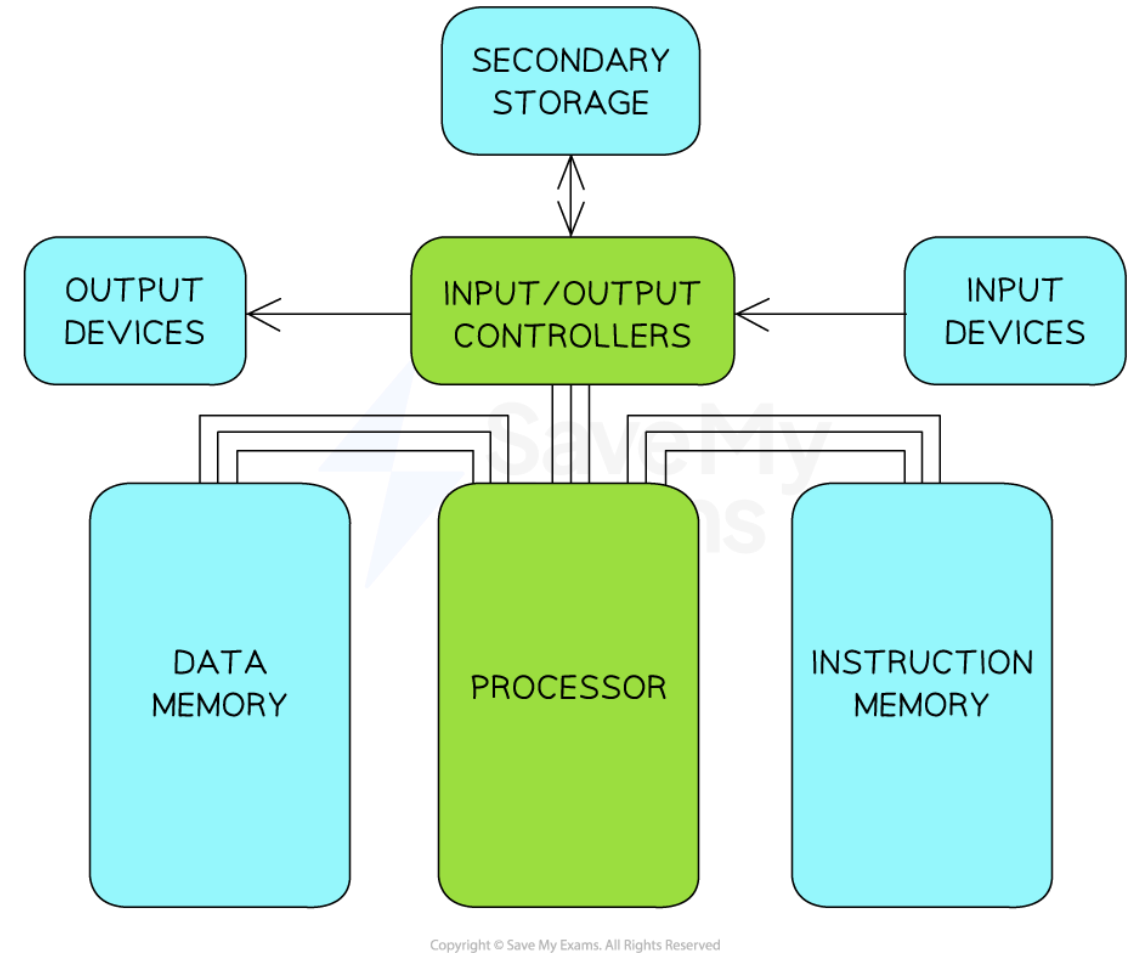


Схематичное изображение Гарвардской архитектуры

Архитектура фон Неймана vs Гарвардская архитектура



Layout of Von Neumann Architecture



Layout of Harvard Architecture

Архитектура фон Неймана vs Гарвардская архитектура

- Гарвардская архитектура эффективна для встраиваемых систем.
- Архитектура фон Неймана — для универсальных компьютеров.

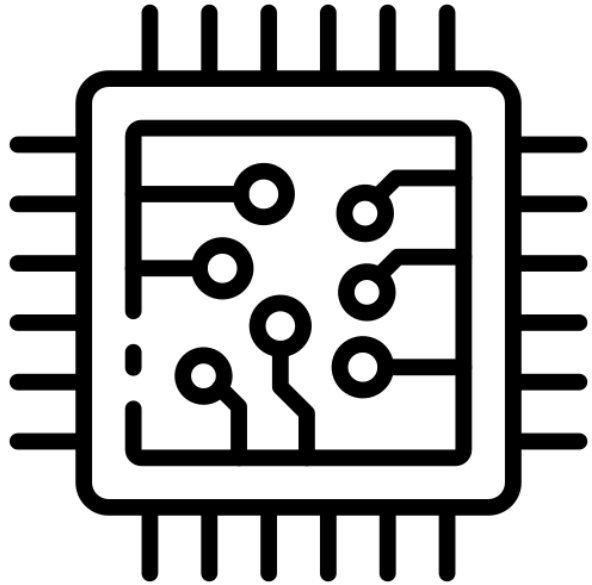
Область применения	Гарвардская архитектура	Архитектура фон Неймана
Микроконтроллеры (MCU)	✓ Доминирует (PIC, AVR, Cortex-M)	✗ Не применяется
Цифровые сигнальные процессоры (DSP)	✓ Доминирует (TMS320, SHARC)	✗ Не применяется
Системы реального времени (RTOS)	✓ Предпочтительна (автомобили, авионика)	⚠ Применяется, но с риском
Персональные компьютеры и серверы	✗ Не применяется	✓ Доминирует (Intel, AMD)
Смартфоны и планшеты	✗ Не применяется	✓ Доминирует (ARM, Apple Silicon)
Суперкомпьютеры	✗ Не применяется	✓ Доминирует (Xeon, EPYC)
Облачные вычисления	✗ Не применяется	✓ Доминирует

Основные различия: Структура и Принципы

Характеристика	Гарвардская архитектура	Архитектура фон Неймана (Принстонская)
Память	Физически разделена: отдельная память для команд (обычно Flash/ROM) и отдельная для данных (обычно RAM).	Единая память: команды и данные хранятся в одном адресном пространстве (обычно RAM).
Шины	Раздельные шины: отдельная шина для доступа к командам и отдельная для данных.	Единая системная шина: одна шина для передачи и команд, и данных.
Производительность	Выше в специализированных задачах: процессор может одновременно выбирать команду и читать/писать данные.	Ниже из-за "узкого места": процессор не может одновременно обращаться к команде и данным — это создает задержку.
Безопасность и надежность	Выше: код в ПЗУ защищен от случайного или злонамеренного изменения.	Ниже: отсутствие разделения кода и данных делает систему уязвимой к атакам (например, переполнение буфера).
Гибкость	Ниже: сложнее реализовать динамическую загрузку кода, JIT-компиляцию.	Выше: программа может динамически изменять себя , загружать модули, что критично для универсальных ОС.
Сложность и стоимость	Выше (в чистом виде): требуется больше выводов, отдельные контроллеры памяти.	Ниже: проще и дешевле реализовать, особенно на ранних этапах развития электроники.

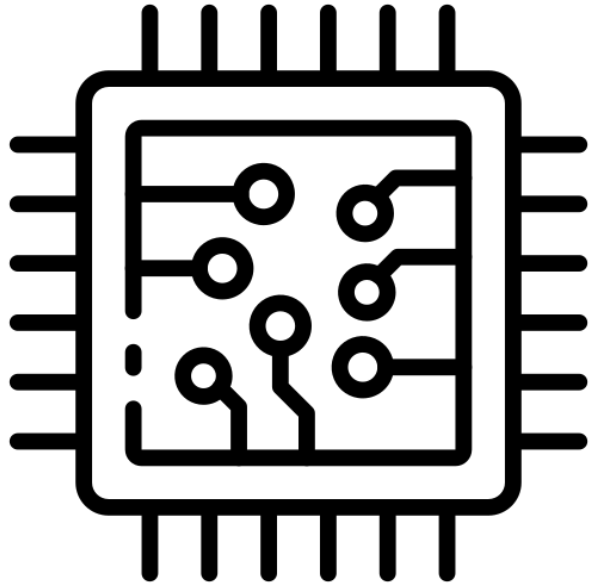
Итог

- Выбор между Гарвардской и фон Неймановской архитектурой – это **компромисс между специализацией и универсальностью**.
 - **Гарвардская архитектура** – это выбор для "встраиваемого мира", где миллиарды устройств должны работать надежно, энергоэффективно и предсказуемо. Она идеальна для задач с жесткими требованиями к времени реакции и безопасности.
 - **Архитектура фон Неймана** – это выбор для "мира универсальных вычислений", где главным требованием является гибкость, способность запускать любое ПО и поддерживать сложные операционные системы. Она — основа для ПК, серверов и смартфонов.
- Современная реальность показывает, что **чистых реализаций почти не существует**. Даже самые "фон-неймановские" процессоры используют гарвардские принципы на уровне кэшей, а "гарвардские" микроконтроллеры часто имеют механизмы для выполнения кода из RAM.
- Это свидетельствует о том, что лучшие идеи обеих архитектур успешно дополняют друг друга, создавая оптимальные решения для конкретных задач.



Классификация Архитектур ЭВМ





Архитектура ЭВМ. По организации памяти и кода



Классификация По организации памяти и кода

- Архитектура ЭВМ по организации памяти и кода классифицируется прежде всего по фундаментальному принципу хранения и доступа к инструкциям и данным.
- На одном полюсе находится **архитектура фон Неймана**, где программа и данные сосуществуют в едином адресном пространстве и передаются по одной общей системной шине.
- Эта модель обеспечивает максимальную универсальность и гибкость — программа может динамически изменять себя, что лежит в основе всей современной программной экосистемы (операционные системы, виртуальные машины, JIT-компиляция).
- Однако ее главный недостаток — «узкое место фон Неймана» — возникает из-за невозможности одновременного доступа к команде и операнду, что ограничивает производительность.
- На противоположном полюсе — **гарвардская архитектура**, физически разделяющая память и шины для команд и данных, что позволяет процессору работать параллельно и значительно повышает скорость и надежность (код защищен от случайной перезаписи).
- Эта модель доминирует в специализированных системах: микроконтроллерах (AVR, PIC) и цифровых сигнальных процессорах (DSP).
- Современные универсальные процессоры (x86, ARM) используют **гибридный подход**: на уровне кэш-памяти первого уровня (L1) они следуют гарвардской модели (раздельные кэши команд и данных), а на уровне системной памяти сохраняют единую фон-неймановскую адресацию, сочетая производительность с гибкостью.

Классификация По организации памяти и кода

- Дальнейшая классификация описывает структуру адресного пространства и способы доступа к памяти в сложных системах.
- **Единое (flat) адресное пространство** предоставляет программисту линейный массив памяти, что упрощает программирование, в то время как сегментированное делит память на логические блоки (код, данные, стек), что было характерно для ранних архитектур, таких как x86 в реальном режиме.
- В многопроцессорных системах ключевым различием является способ доступа к памяти: **UMA (Uniform Memory Access)** обеспечивает всем процессорам равный и быстрый доступ к общей памяти, но плохо масштабируется; **NUMA (Non-Uniform Memory Access)** распределяет память по узлам, обеспечивая высокую масштабируемость ценой неравномерного времени доступа.
- **CC-NUMA** добавляет к NUMA аппаратную когерентность кэшей, упрощая программирование.
- Для крупномасштабных систем используются **распределенная память (кластеры, MPP)**, где каждый узел имеет свою изолированную память, и обмен происходит через сеть (MPI), а также **DSM (Distributed Shared Memory)**, создающая иллюзию единой памяти поверх распределенной.
- Критически важной является **виртуальная память** (страничная, сегментная), которая абстрагирует физическую память, обеспечивая изоляцию процессов и возможность использования дискового пространства.
- Перспективные направления, такие как **Near-/In-Memory Computing (PIM)** и **унифицированная память CPU-GPU (hUMA)**, стремятся преодолеть классическое разделение вычислений и памяти, интегрируя их для повышения производительности и энергоэффективности.

По организации памяти и кода (укрупненно)

- **Фон Неймана**: единая память для инструкций и данных.
- **Гарвардская**: отдельные памяти/шины для инструкций и данных.
- **Модифицированная гарвардская**: общее адресное пространство, но отдельные кэши I/D.
- **Единое адресное пространство** (flat) vs **сегментированное** (segmented).
- **UMA** (Uniform Memory Access): равномерный доступ ко всем ячейкам памяти.
- **NUMA** (Non-Uniform Memory Access): разное время доступа в зависимости от узла.
- **СС-NUMA**: NUMA с когерентностью кэшей.
- **Распределённая память** (cluster/distributed): память локальна узлам, обмен по сети.
- **Виртуальная память**: страничная/сегментно-страничная, TLB.
- и др.

По организации памяти и кода

- **Архитектура фон Неймана (Принстонская):**

- **Единая память** для хранения и команд (кода программы), и данных.
- **Единая системная шина** для доступа к памяти.
- **Преимущества:** Универсальность, гибкость (программа может менять себя), простота программной модели.
- **Недостатки: "Узкое место фон Неймана"** — невозможность одновременного доступа к команде и данным, что ограничивает производительность. Уязвимость безопасности (код и данные не разделены).
- **Применение:** Основа для подавляющего большинства универсальных компьютеров (ПК, серверы, смартфоны).

- **Гарвардская архитектура:**

- **Физически раздельная память для команд и данных.**
- **Раздельные шины** для доступа к каждой из них.
- **Преимущества:** Высокая производительность (параллельный доступ к коду и данным), повышенная надежность и безопасность (код защищен от случайной перезаписи).
- **Недостатки:** Сложнее и дороже в реализации, менее гибкая.
- **Применение:** Микроконтроллеры (AVR, PIC), цифровые сигнальные процессоры (DSP).

- **Модифицированная гарвардская архитектура (Гибридная):**

- **На уровне кэш-памяти L1** — раздельные кэши для инструкций (L1I) и данных (L1D) — это гарвардский принцип.
- **На уровне системной памяти (DRAM) и адресного пространства** — единое, как у фон Неймана.
- **Преимущества:** Комбинирует производительность Гарвардской (параллелизм на уровне кэша) с гибкостью фон Неймана (единое адресное пространство).
- **Применение:** Стандарт де-факто для всех современных высокопроизводительных процессоров (Intel Core, AMD Ryzen, Apple Silicon, ARM Cortex-A).

По организации памяти и кода

- **Единое (flat) адресное пространство**

- Все ячейки памяти (код, данные, стек, устройства) доступны через единый непрерывный диапазон адресов. **Вся память** (ОЗУ, ПЗУ, порты ввода-вывода) **представлена как один непрерывный линейный массив байтов**. Каждый байт имеет уникальный адрес.
- Упрощает программирование и управление памятью.
- **Противоположность**: сегментированная модель.
- **Плюсы**: Простота программирования и управления памятью.
- **Минусы**: Может быть неэффективно для очень больших объемов памяти или систем с разнородными устройствами.
- **Применение**: Современные 32/64-битные системы с виртуальной памятью. Используется в современных ОС и процессорах.

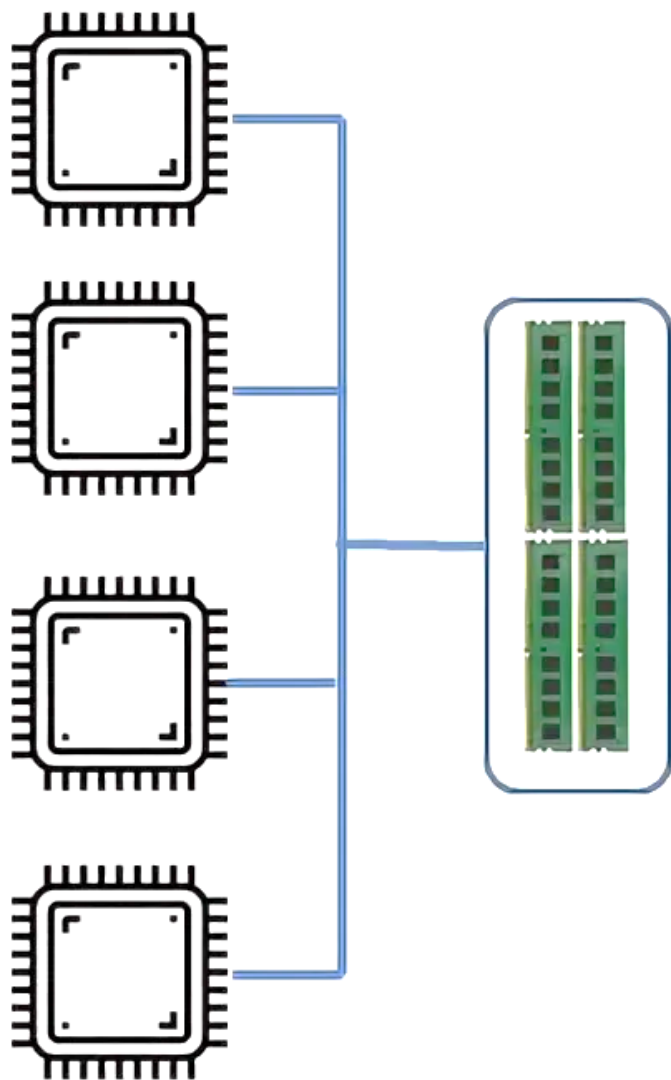
По организации памяти и кода

- **Сегментированное адресное пространство**

- **Память делится на сегменты** (например, сегмент кода, сегмент данных, сегмент стека).
- Адрес состоит из селектора сегмента и смещения.
- Позволяет изолировать области памяти, но усложняет адресацию.
- **Плюсы:** Позволяет эффективно использовать 16-битные регистры для адресации больших объемов памяти (как в x86 в реальном режиме). Упрощает изоляцию кода, данных и стека.
- **Минусы:** Сложность программирования, фрагментация памяти.
- **Применение:** Ранние процессоры (Intel 8086), режим совместимости в x86-64. Пример: x86 в реальном режиме и защищённом режиме (до 64-битного режима).

По организации памяти и кода

UMA



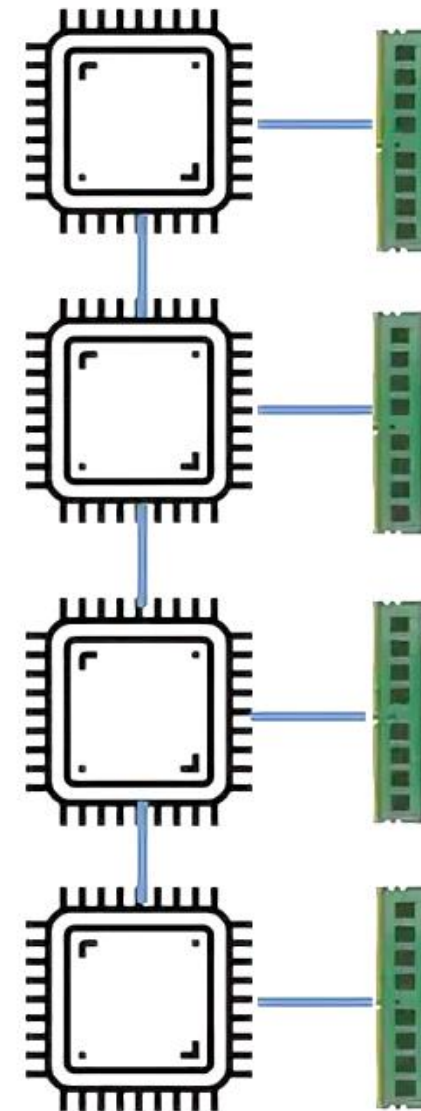
- **UMA (Uniform Memory Access)**

- **Однородный доступ к памяти.**
- Все процессоры в многопроцессорной системе имеют одинаковое время доступа к любой ячейке ОЗУ.
- Общая шина или кроссбар-коммутатор.
- Простота, но масштабируемость ограничена.
- Пример: SMP-системы с общей памятью.

- **NUMA (Non-Uniform Memory Access)**

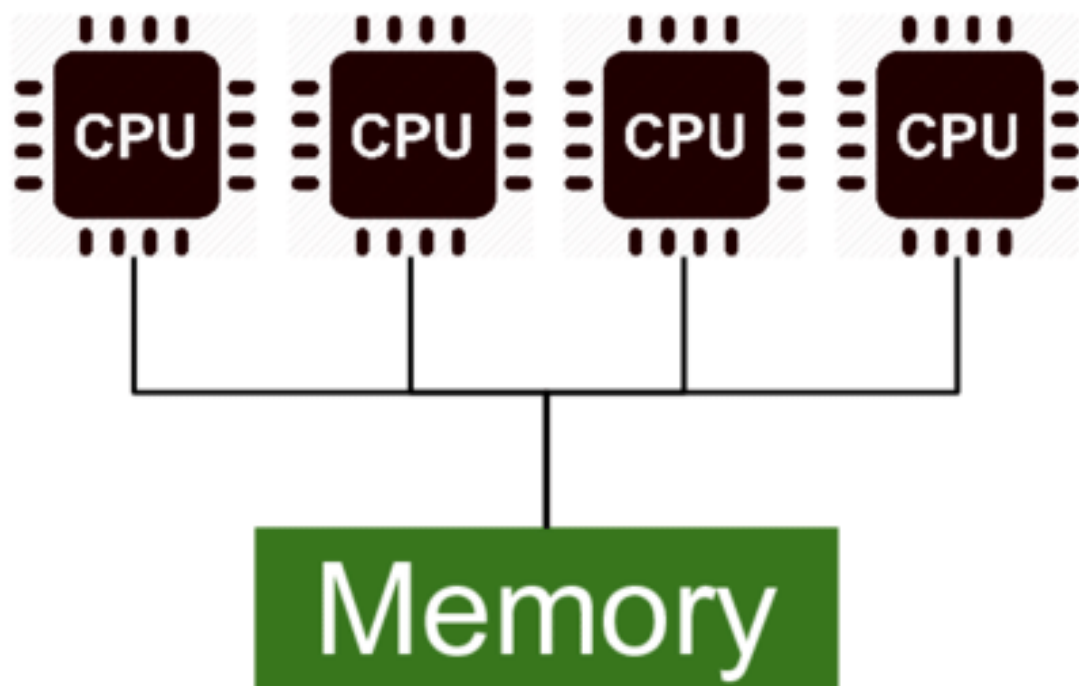
- **Неоднородный доступ к памяти.**
- Память логически едина, но физически распределена по узлам.
- Доступ к "локальной" памяти быстрее, чем к "удалённой".
- Используется в многопроцессорных серверах.
- Пример: системы с несколькими CPU-сокетами (Intel Xeon, AMD EPYC).

NUMA



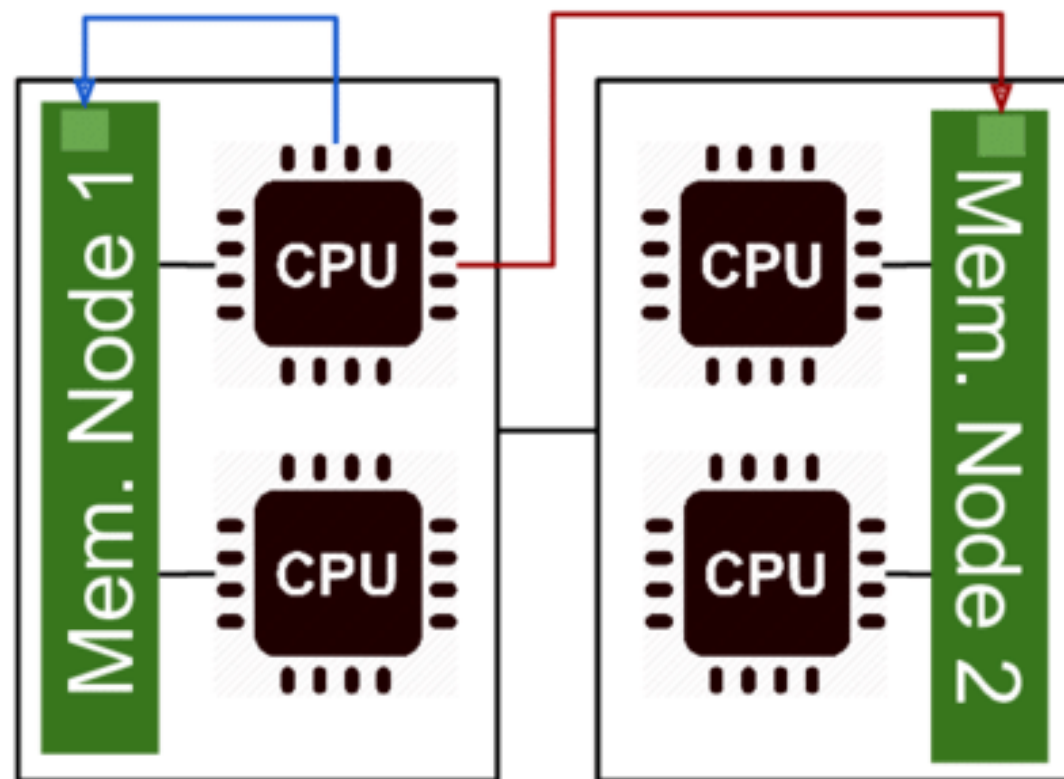
По организации памяти и кода

Uniform Memory Access



Non-Uniform Memory Access

Local Access Remote Access

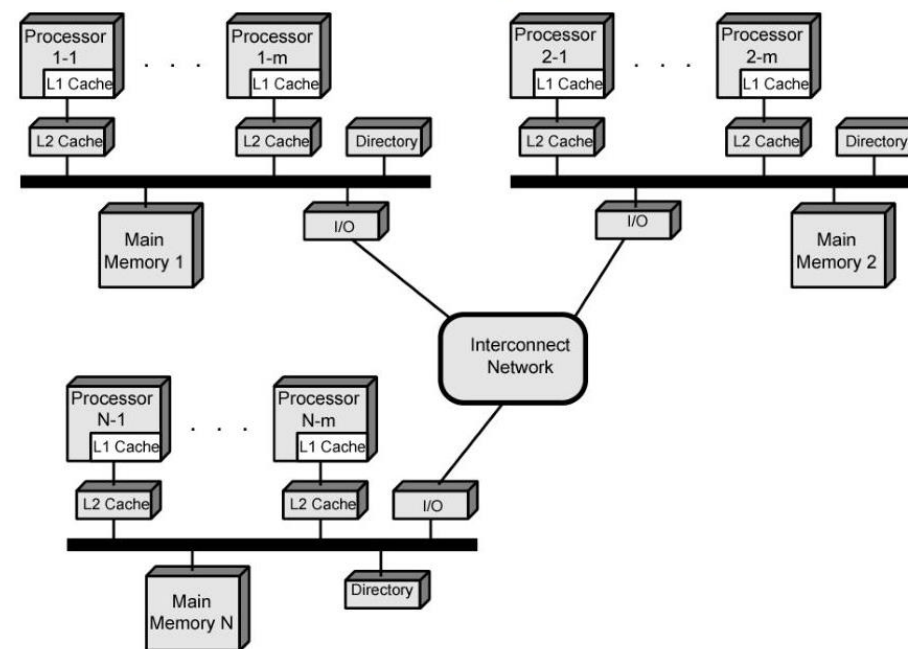


По организации памяти и кода

• CC-NUMA (Cache-Coherent NUMA)

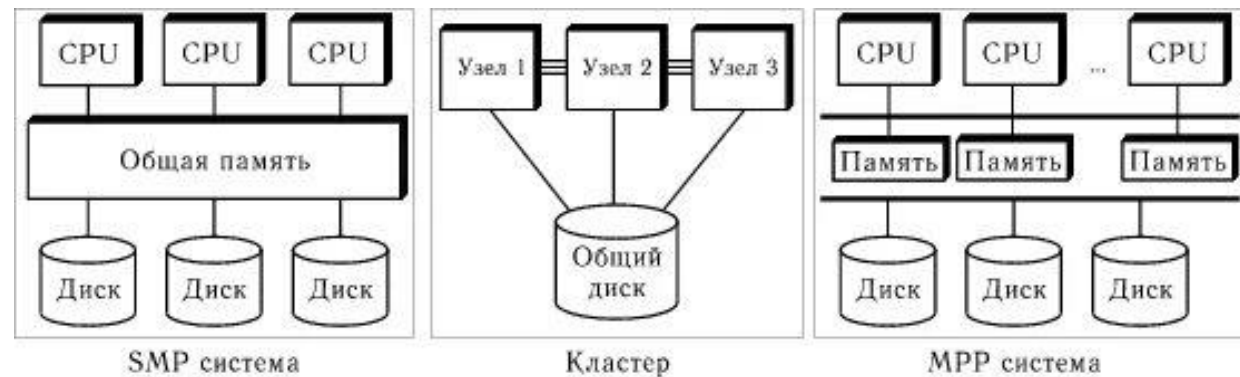
- NUMA с когерентностью кэшей.
- Обеспечивает согласованность данных в кэшах разных процессоров.
- Использует протоколы когерентности (MESI, MOESI).
- Позволяет программам работать с общей памятью, как в UMA, но с NUMA-производительностью.
- Пример: современные серверные архитектуры (AMD EPYC, Intel Scalable Processors).

CC-NUMA Organization



• Распределённая память (cluster, MPP)

- Каждый узел (нода) имеет свою собственную память.
- Нет единого адресного пространства — обмен данными через сети (MPI, RDMA).
- MPP (Massively Parallel Processor) — системы с тысячами процессоров (суперкомпьютеры).
- Высокая масштабируемость, но сложное программирование.
- Пример: кластеры, суперкомпьютеры (типа "Ломоносов").

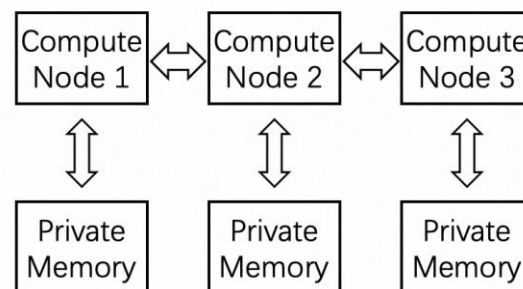


По организации памяти и кода

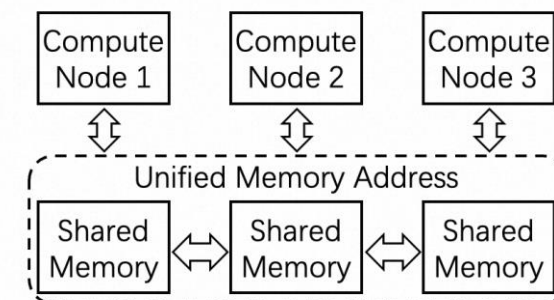
• DSM (Distributed Shared Memory)

- Распределённая разделяемая память.
- Создаёт иллюзию единого адресного пространства на системе с распределённой памятью.
- Реализуется:
 - Hard DSM: аппаратно (специальные коммутаторы, когерентные интерконнекты).
 - Soft DSM: программно (через ОС или среду выполнения).
- Цель — упростить программирование параллельных систем.
- Пример: InfiniBand + RDMA, некоторые HPC-системы.

Message Passing (MP)

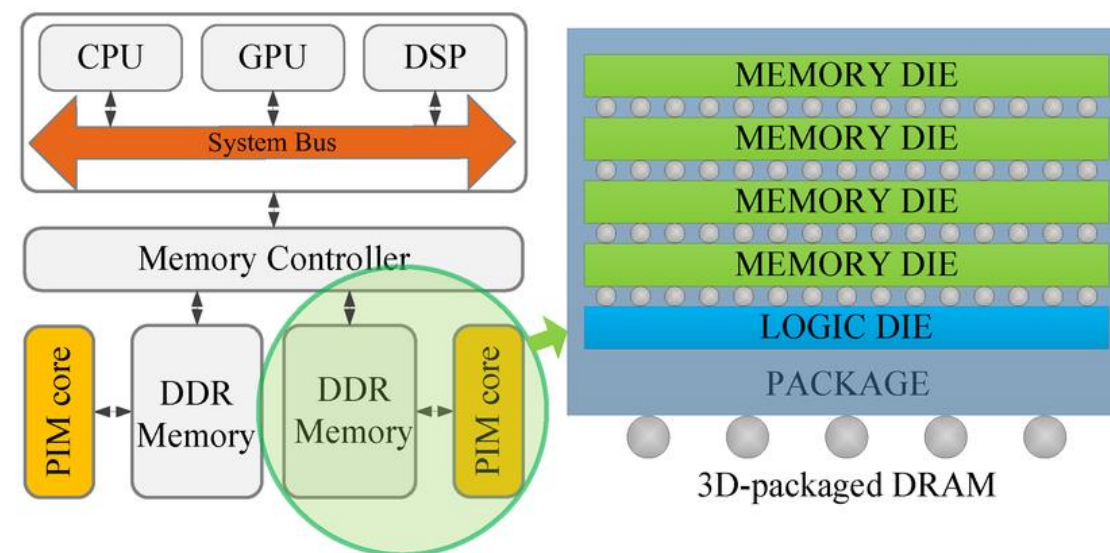


Distributed Shared Memory (DSM)

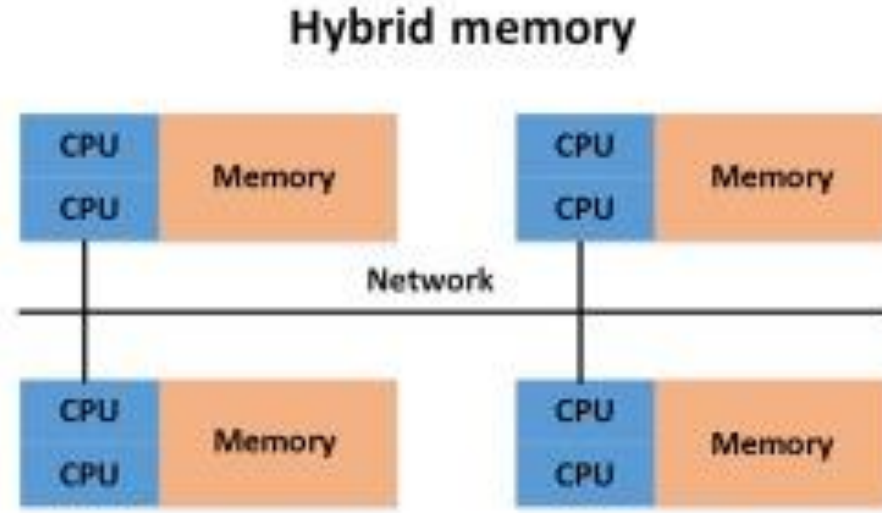
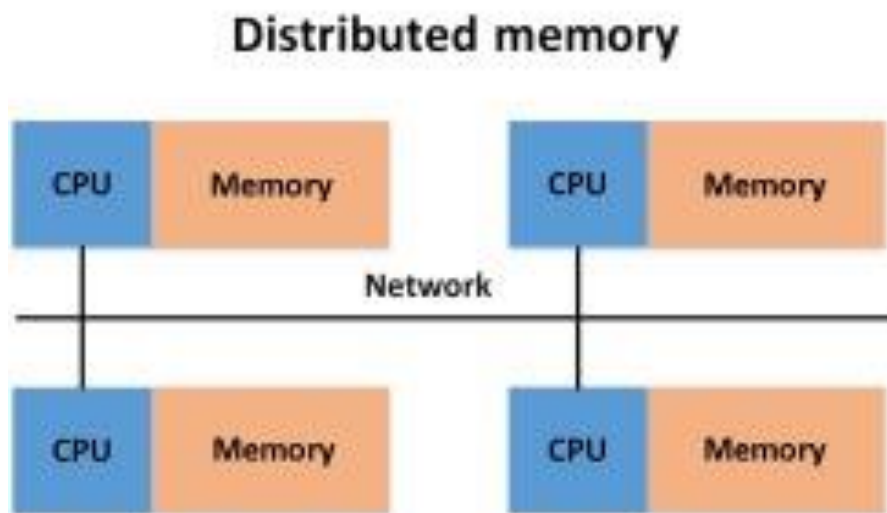
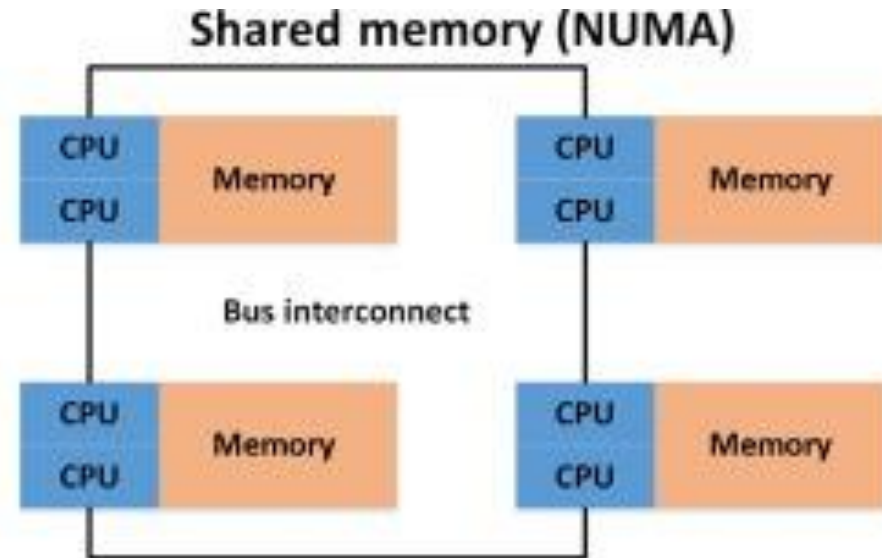
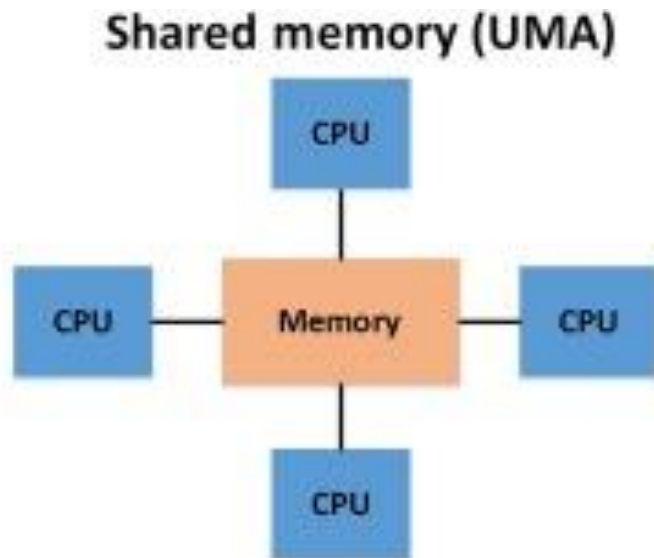


• Near-/In-Memory Computing (PIM — Processing-in-Memory)

- Цель: преодолеть "узкое место памяти" за счёт вычислений рядом с памятью или внутри памяти.
- Near-Memory: процессор и память близко (например, 3D-stacked HBM + логика).
- In-Memory Computing: вычисления выполняются прямо в ячейках памяти (например, с помощью RRAM, memristors).
- Применение: ИИ, нейросети, big data.
- Пример: Samsung HBM-PIM, Intel Optane с вычислительными возможностями.



По организации памяти и кода

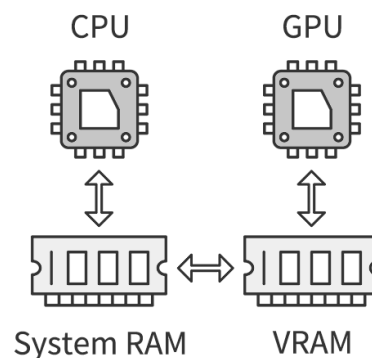


По организации памяти и кода

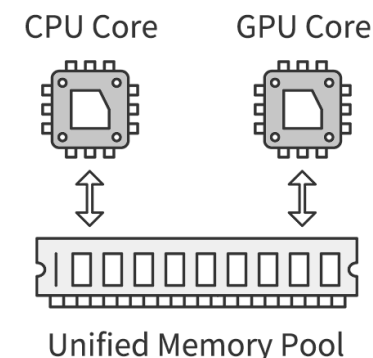
- **Унифицированная память CPU–GPU (UMA в SoC, hUMA / HSA)**

- **UMA (Unified Memory Architecture):**

- CPU и GPU разделяют одну физическую память (в SoC: смартфоны, APU).
- Упрощает обмен данными между CPU и GPU.



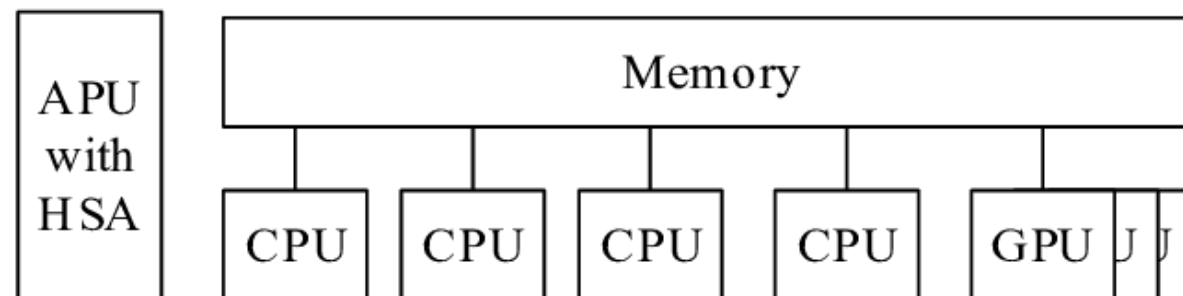
Traditional Memory Architecture



Unified Memory Architecture

- **hUMA (heterogeneous Uniform Memory Access):**

- Часть инициативы HSA (Heterogeneous System Architecture).
- CPU и GPU имеют единое виртуальное адресное пространство.
- Поддержка когерентности кэшей между CPU и GPU.
- Позволяет GPU напрямую обращаться к данным CPU и наоборот.
- Пример: AMD APU (Ryzen с Vega), некоторые ARM Mali-системы.



По организации памяти и кода

- **Виртуальная память**

- Позволяет программам использовать "виртуальные" адреса, независимо от физической памяти.

- **Типы:**

- **Страничная (paging):**

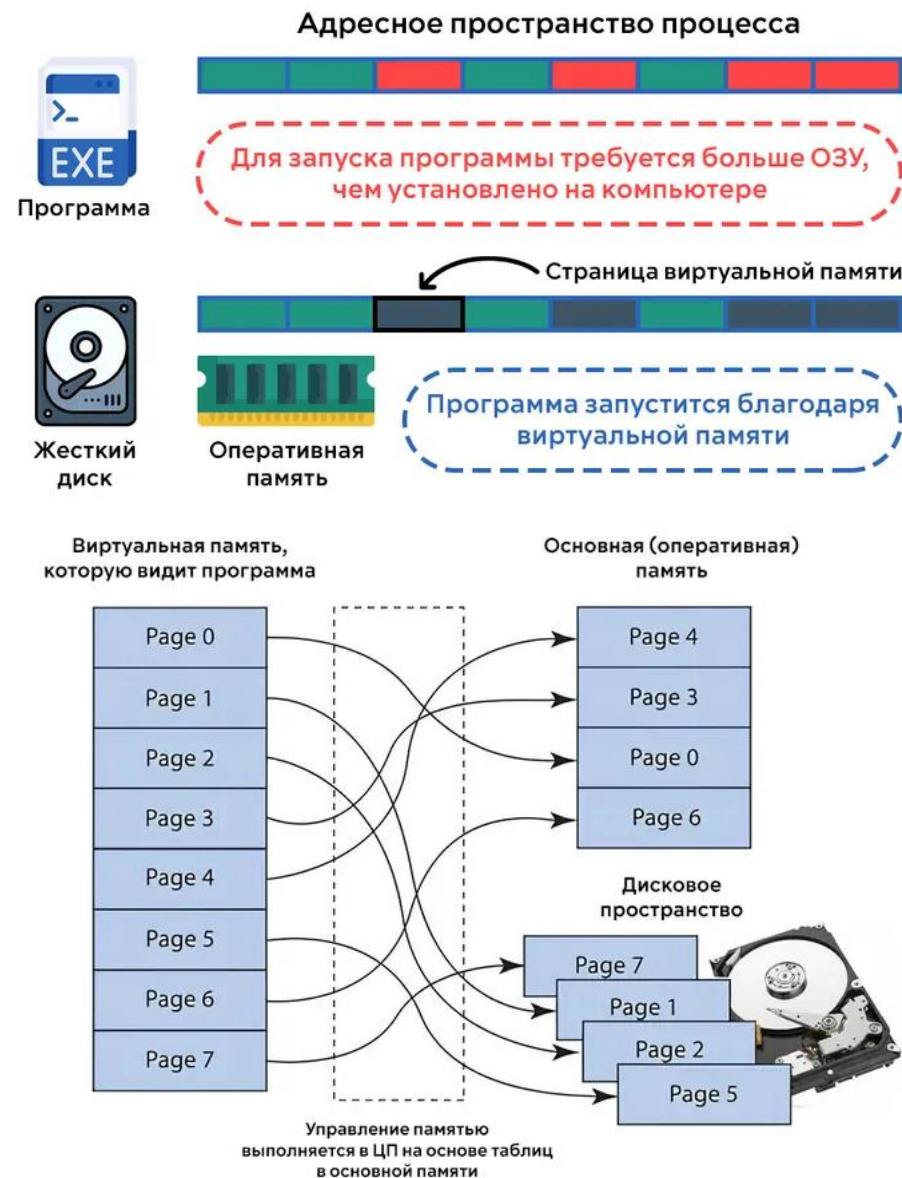
- Память делится на фиксированные страницы (обычно 4 КБ).
- Виртуальные адреса транслируются в физические через таблицы страниц (MMU).
- Поддерживает подкачку на диск (swap).
- Используется везде: Linux, Windows, macOS.

- **Сегментная (segmentation):**

- Память делится на сегменты переменной длины (код, данные, стек).
- Каждый сегмент имеет базовый адрес и длину.
- Позволяет гибкое управление, но сложна в реализации.
- Пример: x86 в защищённом режиме (устаревшее).

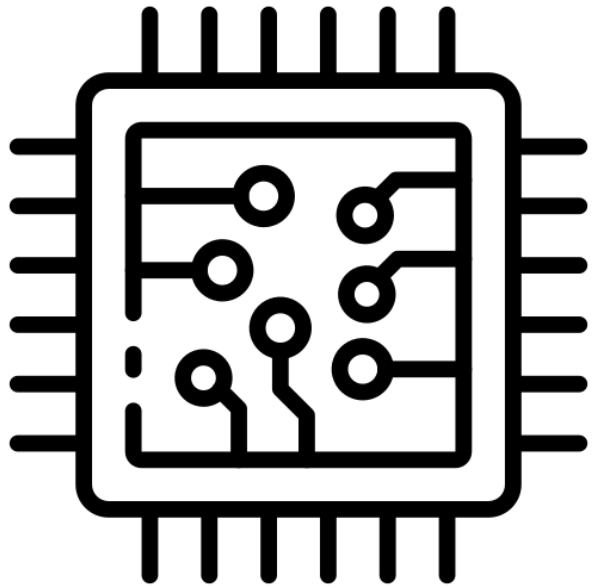
- **Странично-сегментная:**

- Комбинация: сегменты делятся на страницы.
- Позволяет извлечь преимущества обоих подходов.
- Пример: IBM System/360, некоторых старых ОС.



По организации памяти и кода

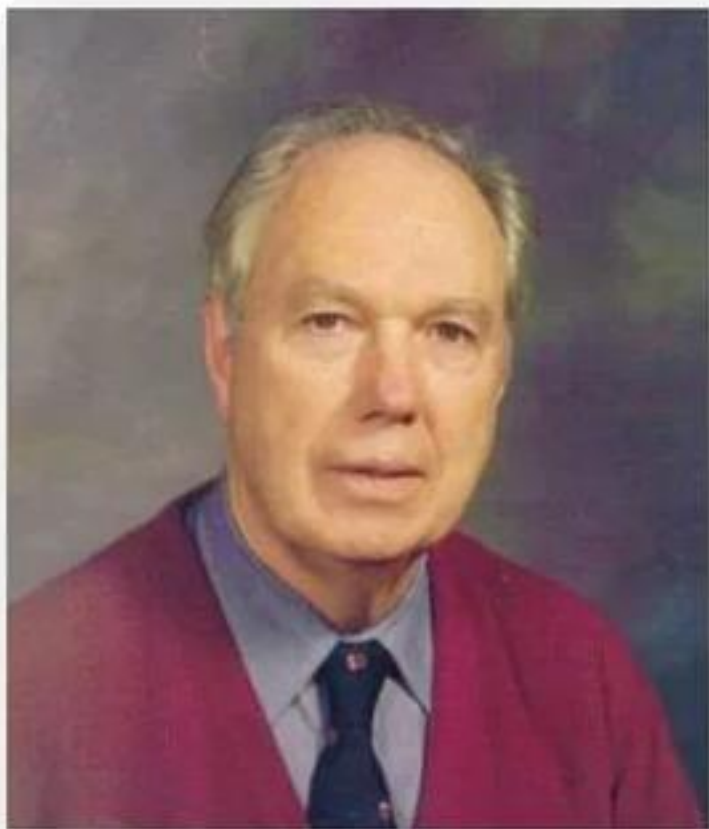
Концепция	Ключевая особенность	Где используется
Фон Неймана	Единая память для кода и данных	Универсальные CPU
Гарвардская	Раздельные память и шины	Микроконтроллеры, DSP
Модифицированная Гарвардская	L1I/L1D разделены, единое адресное пространство	x86, ARM, RISC-V
Flat memory	Линейное адресное пространство	Современные ОС
Сегментированная память	Адрес = сегмент + смещение	x86 (режимы 16/32 бит)
UMA	Одинаковый доступ к памяти	SMP, десктопы
NUMA	Неоднородный доступ	Многосокетные серверы
CC-NUMA	NUMA + когерентность кэшей	Серверы, HPC
Распределённая память	Нет общей памяти	Кластеры, MPP
DSM	Виртуальная общая память	HPC, распределённые системы
Виртуальная память	Абстракция физической памяти	Все современные ОС
PIM / In-Memory	Вычисления в памяти	ИИ, ускорители
hUMA / HSA	Общая виртуальная память CPU-GPU	APU, SoC, GPGPU



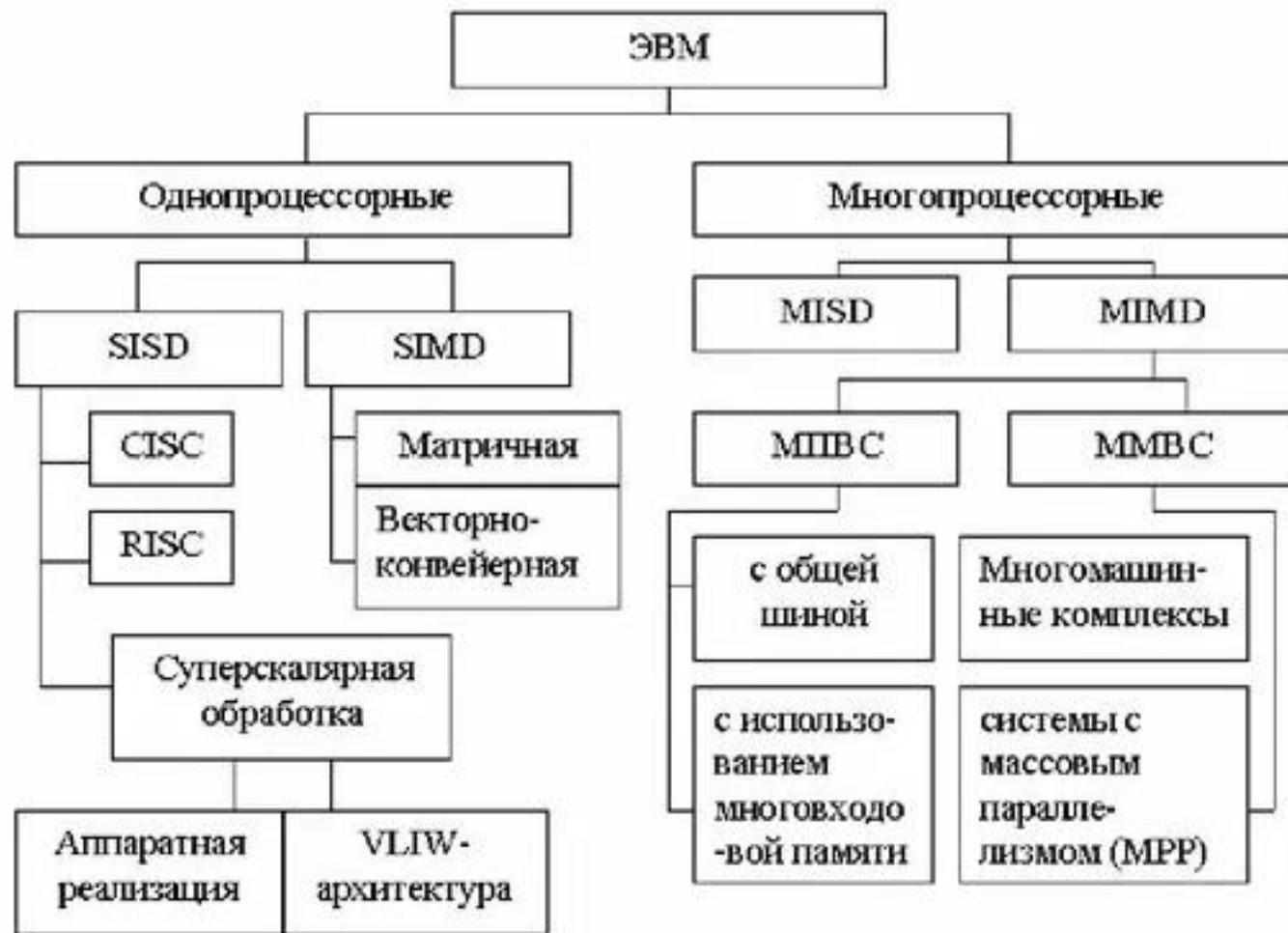
Архитектура ЭВМ. По модели параллелизма (классификация Флинна)



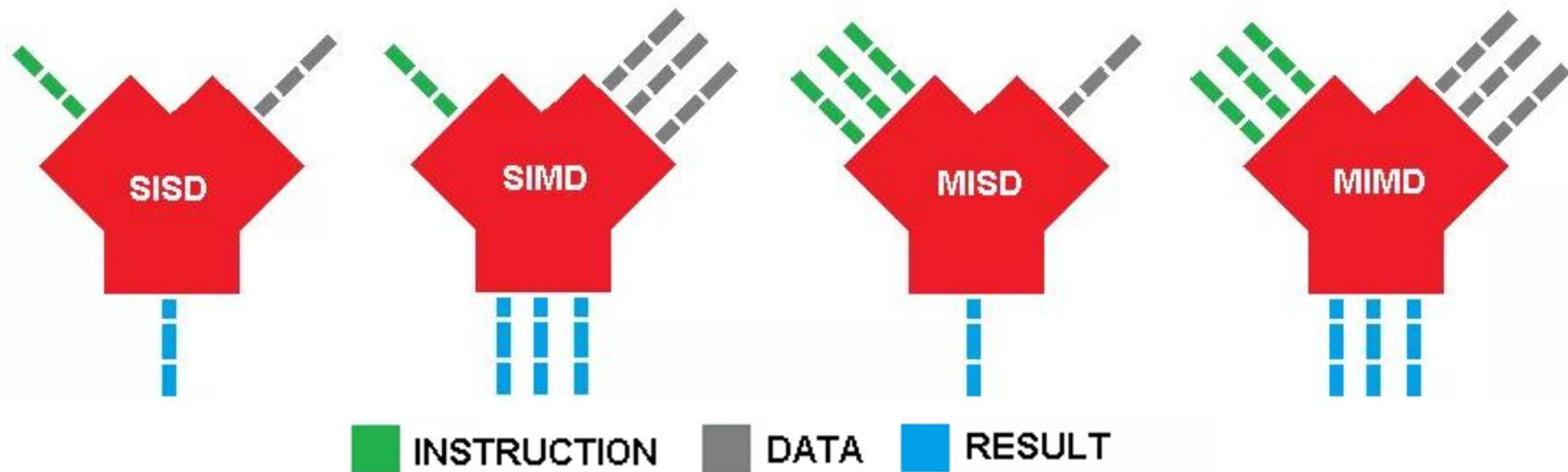
Классификация Флинна



Майкл Флинн



Классификация Флинна



В зависимости от количества потоков команд и потоков данных Флинн выделяет четыре класса архитектур: SISD, MISD, SIMD, MIMD.

SISD (Single Instruction, Single Data): один поток инструкций, один поток данных.

SIMD (Single Instruction, Multiple Data): один поток инструкций, много потоков данных (векторные процессоры, расширения SSE/AVX, NEON; классические векторные Cray).

MISD (Multiple Instruction, Single Data): много потоков инструкций, один поток данных (редко; примеры – системы с избыточностью для отказоустойчивости).

MIMD (Multiple Instruction, Multiple Data): много потоков инструкций и данных (многопроцессорные и многоядерные системы).

Классификация Флинна

Single data stream

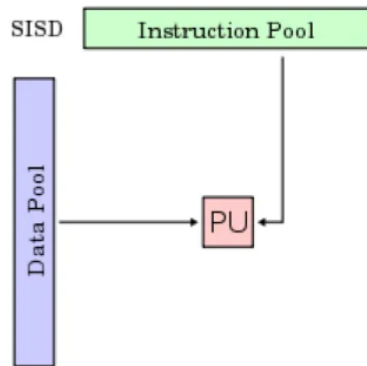
Multiple data streams

Single instr. stream

SISD: фон-Неймановская архитектура.

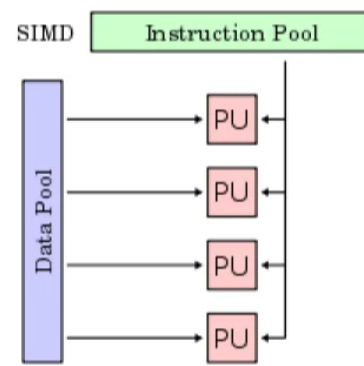
Один процессор выполняет один поток команд, оперируя одним потоком данных.

В каждый момент времени выполняется лишь одна операция над одним элементом данных (числовым или каким-либо другим значением).



SIMD: параллелизм на уровне данных.

SIMD-компьютеры состоят из **одного командного процессора** (управляющего модуля), называемого контроллером, и **нескольких модулей обработки данных**, называемых процессорными элементами. Управляющий модуль принимает, анализирует и выполняет команды. Если в команде встречаются данные, контроллер рассылает на все процессорные элементы команду, и эта команда выполняется на нескольких или на всех процессорных элементах. Каждый процессорный элемент имеет свою собственную память для хранения данных. В SIMD компьютере управление выполняется контроллером, а «арифметика» отдана процессорным элементам.

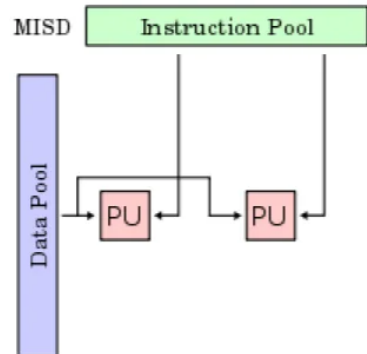


Multiple instr. streams

MISD: систолический массив процессоров, конвейерная архитектура — т.к. данные будут различаться после обработки на каждой стадии в конвейере.

Несколько функциональных модулей (два или более) выполняют различные операции над одними данными.

Отказоустойчивые компьютеры, выполняющие одни и те же команды избыточно с целью обнаружения ошибок.

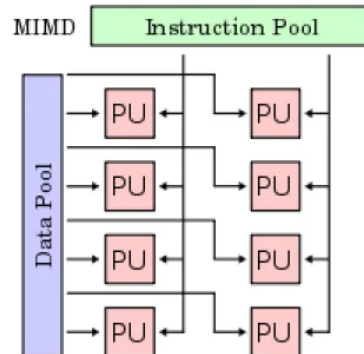


MIMD: параллелизм вычислений.

Несколько процессоров, которые функционируют асинхронно и независимо. В любой момент, различные процессоры могут выполнять различные команды над различными частями данных. MIMD-машины могут быть либо с общей памятью, либо с распределяемой памятью.

Обработка разделена на несколько потоков, каждый с собственным аппаратным состоянием процессора, в рамках единственного определённого программным обеспечением процесса или в пределах множественных процессов. Поскольку система имеет несколько потоков, ожидающих выполнения (системные или пользовательские потоки), эта архитектура эффективно использует аппаратные ресурсы.

В MIMD могут возникнуть проблемы взаимной блокировки и состязания за обладание ресурсами, так как потоки, пытаясь получить доступ к ресурсам, могут столкнуться непредсказуемым способом.



Классификация Флинна (SISD, SIMD, MISD, MIMD) — базовая модель классификации архитектур.

SPMD, SIMT, MPMD — современные модели программирования и исполнения, расширяющие или уточняющие классификацию Флинна.

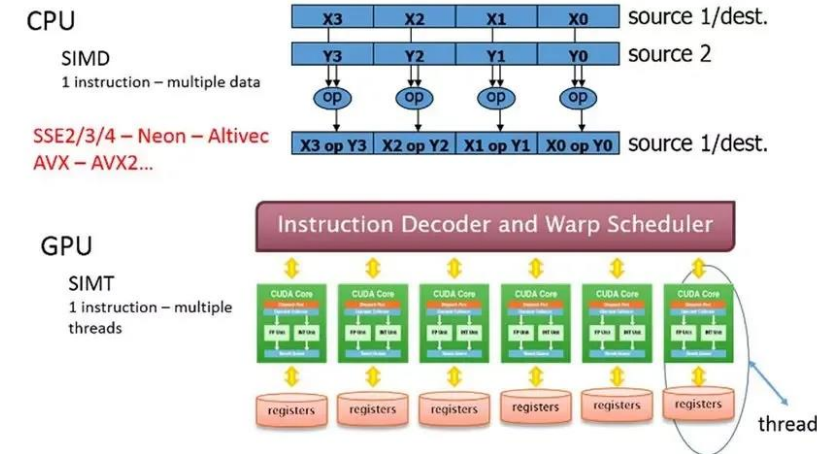
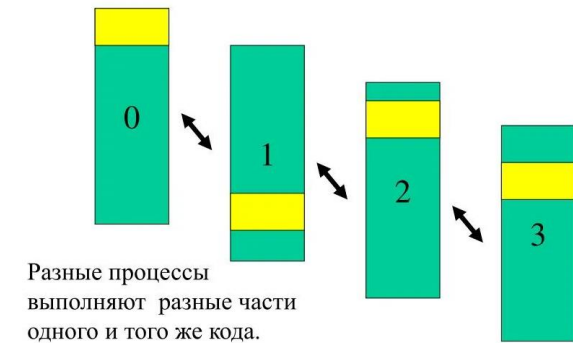
SIMT — фактически реализует SIMD-подобное поведение, но с семантикой многопоточности.

SPMD — наиболее частая модель в параллельных вычислениях, реализуемая на MIMD-архитектурах.

По модели параллелизма

- **SPMD (Single Program, Multiple Data)** – программная модель, при которой одна и та же программа параллельно выполняется на разных процессорах или ядрах с разными данными. Широко используется в параллельных и распределённых системах (MPI, OpenMP, GPU-ядра). **SPMD** — стандарт для параллельных вычислений: гибкий и масштабируемый.
- **SIMT (Single Instruction, Multiple Thread)** – аппаратная модель выполнения в GPU, где одна инструкция применяется к множеству потоков одновременно. Поддерживает дивергенцию ветвлений (разные потоки могут выполнять разные ветки кода), в отличие от классического SIMD. **SIMT** — ключ к высокой производительности GPU, но чувствителен к структуре кода.
- **MPMD (Multiple Program, Multiple Data)** – модель, в которой разные процессоры или узлы выполняют разные программы на своих наборах данных. Используется в гетерогенных и гибридных HPC-системах (например, CPU + GPU + FPGA). **MPMD** — путь к максимальной эффективности в гетерогенных системах, требует сложного управления.

SPMD-модель.



По модели параллелизма

Модель	Тип	Описание	Примеры использования	Примечания
SISD (Single Instruction, Single Data)	Архитектура	Один поток команд обрабатывает один поток данных. Последовательная обработка.	Классические однопроцессорные ЭВМ, ранние CPU.	Базовый случай, без параллелизма.
SIMD (Single Instruction, Multiple Data)	Архитектура	Одна команда применяется к нескольким данным одновременно.	Векторные процессоры (Cray), SSE/AVX (x86), NEON (ARM), GPU (на уровне warp/wavefront).	Высокая производительность для регулярных данных. Не поддерживает дивергенцию ветвлений.
MISD (Multiple Instruction, Single Data)	Архитектура	Несколько команд обрабатывают один поток данных.	Системы с избыточностью (например, отказоустойчивые: три модуля обрабатывают один сигнал).	Теоретически редкая модель, используется в специализированных системах.
MIMD (Multiple Instruction, Multiple Data)	Архитектура	Несколько потоков команд и данных. Независимое выполнение.	Многоядерные процессоры, многопроцессорные системы, кластеры, SMP, NUMA.	Наиболее распространённая модель в современных многопроцессорных системах.
SPMD (Single Program, Multiple Data)	Программная модель	Одна программа запускается на нескольких узлах/потоках, но работает с разными данными.	MPI-приложения, параллельные вычисления на CPU/GPU.	Частный случай MIMD. Поддерживает условные ветвления.
SIMT (Single Instruction, Multiple Thread)	Аппаратная модель	Расширение SIMD: одна инструкция — много потоков, но с возможностью дивергенции ветвлений.	GPU (NVIDIA CUDA, AMD HIP, OpenCL).	Аппаратно реализуется на GPU. Потоки группируются в warps/wavefronts.
MPMD (Multiple Program, Multiple Data)	Программная модель	Разные узлы выполняют разные программы на своих данных.	Гетерогенные HPC-системы (CPU + GPU + FPGA), эмбедед-системы.	Наиболее гибкая, но сложная в управлении модель.